

(4) 访问控制方法不同。

文件传送协议 FTP 只提供文件传送的一些基本的服务，它使用 TCP 可靠的运输服务。FTP 的主要功能是减少或消除在不同操作系统下处理文件的不兼容性。

FTP 使用客户服务器方式。一个 FTP 服务器进程可同时为多个客户进程提供服务。FTP 的服务器进程由两大部分组成：一个主进程，负责接受新的请求；另外有若干个从属进程，负责处理单个请求。

主进程的工作步骤如下：

(1) 打开熟知端口（端口号为 21），使客户进程能够连接上。

(2) 等待客户进程发出连接请求。

(3) 启动从属进程处理客户进程发来的请求。从属进程对客户进程的请求处理完毕后即终止，但从属进程在运行期间根据需要还可能创建其他一些子进程。

(4) 回到等待状态，继续接受其他客户进程发来的请求。主进程与从属进程的处理是并发进行的。

FTP 的工作情况如图 6-5 所示。图中的椭圆圈表示在系统中运行的进程。图中的服务器端有两个从属进程：控制进程和数据传送进程。为简单起见，服务器端的主进程没有画上。客户端除了控制进程和数据传送进程外，还有一个用户界面进程用来和用户接口。

在进行文件传输时，FTP 的客户和服务器之间要建立两个并行的 TCP 连接：“控制连接”和“数据连接”。控制连接在整个会话期间一直保持打开，FTP 客户所发出的传送请求，通过控制连接发送给服务器端的控制进程，但控制连接并不用来传送文件。实际用于传输文件的是“数据连接”。服务器端的控制进程在接收到 FTP 客户发送来的文件传输请求后就创建“数据传送进程”和“数据连接”，用来连接客户端和服务器的数据传送进程。数据传送进程实际完成文件的传送，在传送完毕后关闭“数据传送连接”并结束运行。由于 FTP 使用了一个分离的控制连接，因此 FTP 的控制信息是带外(out of band)传送的。

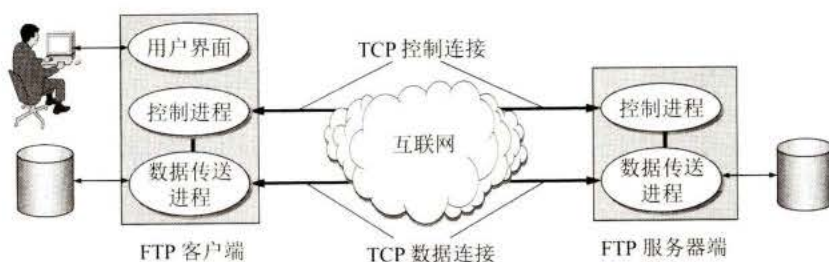


图 6-5 FTP 使用的两个 TCP 连接

当客户进程向服务器进程发出建立连接请求时，要寻找连接服务器进程的熟知端口 21，同时还要告诉服务器进程自己的另一个端口号码，用于建立数据传送连接。接着，服务器进程用自己传送数据的熟知端口 20 与客户进程所提供的端口号建立数据传送连接。由于 FTP 使用了两个不同的端口号，所以数据连接与控制连接不会发生混乱。

使用两个独立的连接的主要好处是使协议更加简单和更容易实现，同时在传输文件时还可以利用控制连接对文件的传输进行控制。例如，客户发送“请求终止传输”。

FTP 并非对所有的数据传输都是最佳的。例如，计算机 A 上运行的应用程序要在远地计算机 B 的一个很大的文件末尾添加一行信息。若使用 FTP，则应先将此文件从计算机 B

传送到计算机 A，添加上这一行信息后，再用 FTP 将此文件传送到计算机 B，来回传送这样大的文件很花时间。实际上这种传送是不必要的，因为计算机 A 并没有使用该文件的内容。

然而网络文件系统 NFS 则采用另一种思路。NFS 允许应用进程打开一个远地文件，并能在该文件的某一个特定的位置上开始读写数据。这样，NFS 可使用户只复制一个大文件中的一个很小的片段，而不需要复制整个大文件。对于上述例子，计算机 A 中的 NFS 客户软件，把要添加的数据和在文件后面写数据的请求一起发送到远地的计算机 B 中的 NFS 服务器，NFS 服务器更新文件后返回应答信息。在网络上传送的只是少量的修改数据。

6.2.3 简单文件传送协议 TFTP

TCP/IP 协议族中还有一个简单文件传送协议 TFTP (Trivial File Transfer Protocol)，它是一个很小且易于实现的文件传送协议[RFC 1350, STD33]。

虽然 TFTP 也使用客户服务器方式，但它使用 UDP 数据报，因此 TFTP 需要有自己的差错改正措施。TFTP 只支持文件传输而不支持交互。TFTP 没有一个庞大的命令集，没有列目录的功能，也不能对用户进行身份鉴别。

TFTP 的主要优点有两个。第一，TFTP 可用于 UDP 环境。例如，当需要将程序或文件同时向许多机器下载时就往往需要使用 TFTP。第二，TFTP 代码所占的内存较小。这对较小的计算机或某些特殊用途的设备是很重要的。这些设备不需要硬盘，只需要固化了 TFTP、UDP 和 IP 的小容量只读存储器即可。当接通电源后，设备执行只读存储器中的代码，在网络上广播一个 TFTP 请求。网络上的 TFTP 服务器就发送响应，其中包括可执行二进制程序。设备收到此文件后将其放入内存，然后开始运行程序。这种方式增加了灵活性，也减少了开销。

TFTP 的主要特点是：

- (1) 每次传送的数据报文中有 512 字节的数据，但最后一次可不足 512 字节。
- (2) 数据报文按序编号，从 1 开始。
- (3) 支持 ASCII 码或二进制传送。
- (4) 可对文件进行读或写。
- (5) 使用很简单的首部。

TFTP 的工作很像停止等待协议（见第 5 章 5.4.1 节）。发送完一个文件块后就等待对方的确认，确认时应指明所确认的块编号。发完数据后在规定时间内收不到确认就要重发数据 PDU。发送确认 PDU 的一方若在规定时间内收不到下一个文件块，也要重发确认 PDU。这样就可保证文件的传送不致因某一个数据报的丢失而告失败。

在一开始工作时。TFTP 客户进程发送一个读请求报文或写请求报文给 TFTP 服务器进程，其熟知端口号码为 69。TFTP 服务器进程要选择一个新的端口和 TFTP 客户进程进行通信。若文件长度恰好为 512 字节的整数倍，则在文件传送完毕后，还必须在最后发送一个只含首部而无数据的数据报文。若文件长度不是 512 字节的整数倍，则最后传送数据报文中的数据字段一定不满 512 字节，这正好可作为文件结束的标志。

6.3 远程终端协议 TELNET

TELNET 是一个简单的远程终端协议[RFC 854, STD8]。用户用

扫一扫



视频讲解

扫一扫



视频讲解

TELNET 就可在其所在地通过 TCP 连接注册（即登录）到远地的另一台主机上（使用主机名或 IP 地址）。TELNET 能将用户的击键传到远地主机，同时也能将远地主机的输出通过 TCP 连接返回到用户屏幕。这种服务是透明的，因为用户感觉到好像键盘和显示器是直接连在远地主机上的。因此，TELNET 又称为终端仿真协议。

TELNET 并不复杂，以前应用得很多。现在由于计算机的功能越来越强，用户已较少使用 TELNET 了，可作为历史资料了解一下即可。

TELNET 也使用客户服务器方式。在本地系统运行 TELNET 客户进程，而在远地主机则运行 TELNET 服务器进程。和 FTP 的情况相似，服务器中的主进程等待新的请求，并产生从属进程来处理每一个连接。

TELNET 能够适应许多计算机和操作系统的差异。例如，对于文本中一行的结束，有的系统使用 ASCII 码的回车(CR)，有的系统使用换行(LF)，还有的系统使用两个字符，回车-换行(CR-LF)。又如，在中断一个程序时，许多系统使用 Control-C (^C)，但也有系统使用 ESC 按键。为了适应这种差异，TELNET 定义了数据和命令应怎样通过互联网。这些定义就是所谓的网络虚拟终端 NVT (Network Virtual Terminal)。图 6-6 说明了 NVT 的意义。客户软件把用户的击键和命令转换成 NVT 格式，并送交服务器。服务器软件把收到的数据和命令从 NVT 格式转换成远地系统所需的格式。向用户返回数据时，服务器把远地系统的格式转换为 NVT 格式，本地客户再从 NVT 格式转换到本地系统所需的格式。

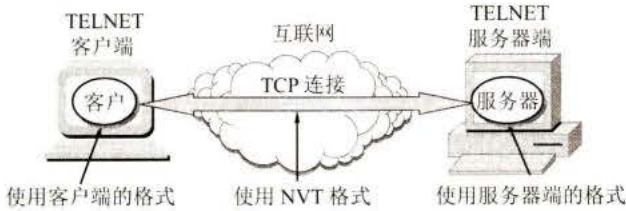


图 6-6 TELNET 使用网络虚拟终端 NVT 格式

NVT 的格式定义很简单。所有的通信都使用 8 位一个字节。在运转时，NVT 使用 7 位 ASCII 码传送数据，而当高位置 1 时用作控制命令。ASCII 码共有 95 个可打印字符（如字母、数字、标点符号）和 33 个控制字符。所有可打印字符在 NVT 中的意义和在 ASCII 码中一样。但 NVT 只使用了 ASCII 码的控制字符中的几个。此外，NVT 还定义了两字符的 CR-LF 为标准的行结束控制符。当用户键入回车按键时，TELNET 的客户就把它转换为 CR-LF 再进行传输，而 TELNET 服务器要把 CR-LF 转换为远地机器的行结束字符。

TELNET 的选项协商(Option Negotiation)使 TELNET 客户和 TELNET 服务器可商定使用更多的终端功能，协商的双方是平等的。

6.4 万维网 WWW

6.4.1 万维网概述

万维网 WWW (World Wide Web)是一个大规模的、联机式的信息储藏所，英文简称为 Web，而不是什么特殊的计算机网络。万维网用链接的方法能非常方便地从互联网上的一个站点访问另一个站点（也就是所谓的“链接到另一个站点”），从而主动地按需获取丰富的信

息。图 6-7 说明了万维网提供分布式服务的特点。

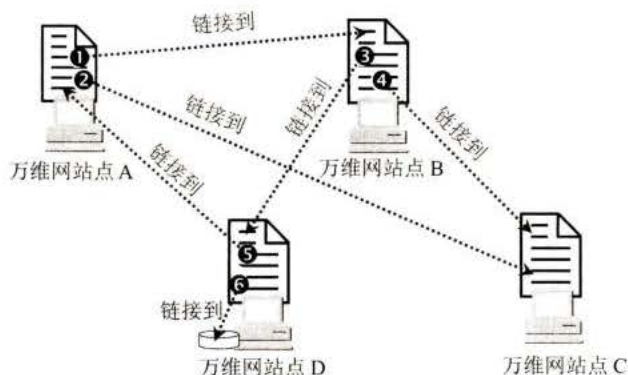


图 6-7 万维网提供分布式服务

图 6-7 画出了四个万维网上的站点，它们可以相隔数千公里，但都必须连接在互联网上。每个万维网站点都存放了许多文档。在这些文档中有一些地方的文字是用特殊方式显示的（例如用不同的颜色，或添加了下划线），而当我们把鼠标移动到这些地方时，鼠标的箭头就变成了一只手的形状。这就表明这些地方有一个**链接(link)**（这种链接有时也称为**超链接(hyperlink)**），如果我们在这些地方点击鼠标左键，就可从这个文档链接到可能相隔很远的另一个文档。经过一定的时延（几秒钟、几分钟甚至更长，取决于所链接的文档的大小和网络的拥塞情况），在我们的屏幕上就能将远方传送过来的文档显示出来。例如，站点 A 的某个文档中有两个地方有链接。点击链接①可链接到站点 B 的某个文档，点击②可链接到站点 C。站点 B 的文档也有两个链接。点击链接③可链接到站点 D，点击链接④可链接到站点 C，但站点 C 的这个文档已无其他的链接了。站点 D 的文档中有两个链接。点击⑤可链接到站点 A，点击⑥可以链接到存储在本站点硬盘中的文档。

正是由于万维网的出现，使计算机的操作发生了革命性的变化。不必在键盘上输入复杂而难以记忆的命令，而改用鼠标点击一下屏幕上的链接，这就使互联网从仅由少数计算机专家使用变为普通百姓也能利用的信息资源。万维网的出现使网站数按指数规律增长，因而成为互联网发展中的一个非常重要的里程碑。

万维网是欧洲粒子物理实验室的 Tim Berners-Lee 最初于 1989 年 3 月提出的。1993 年 2 月，第一个图形界面的浏览器(browser)开发成功，名字叫作 Mosaic。1995 年著名的 Netscape Navigator 浏览器上市。目前流行的浏览器很多，如微软公司的 Internet Explorer（简称 IE），谷歌公司的 Chrome 浏览器，腾讯公司的 QQ 浏览器，苹果公司的 Safari 浏览器，等等。

万维网是一个分布式的**超媒体(hypermedia)**系统，它是**超文本(hypertext)**系统的扩充。所谓超文本是指包含指向其他文档的链接的文本(text)。也就是说，一个超文本由多个信息源链接成，而这些信息源可以分布在世界各地，并且数目也是不受限制的。利用一个链接可使用户找到远在地方的另一个文档，而这又可链接到其他的文档（依此类推）。这些文档可以位于世界上任何一个接在互联网上的超文本系统中。超文本是万维网的基础。

超媒体与超文本的区别是文档内容不同。超文本文档仅包含文本信息，而超媒体文档还包含其他表示方式的信息，如图形、图像、声音、动画以及视频图像等。

分布式的和非分布式的超媒体系统有很大区别。在非分布式系统中，各种信息都驻留

在单个计算机的磁盘中。由于各种文档都可从本地获得，因此这些文档之间的链接可进行一致性检查。所以，一个非分布式超媒体系统能够保证所有的链接都是有效的和一致的。

万维网把大量信息分布在整个互联网上。每台主机上的文档都独立进行管理。对这些文档的增加、修改、删除或重新命名都不需要（实际上也不可能）通知到互联网上成千上万的节点。这样，万维网文档之间的链接就经常会不一致。例如，主机 A 上的文档 X 本来包含了一个指向主机 B 上的文档 Y 的链接。若主机 B 的管理员在某日删除了文档 Y，那么主机 A 的上述链接显然就失效了（但是 B 并没有责任必须通知 A）。

万维网以客户服务器方式工作。上面所说的浏览器就是在用户主机上的万维网客户程序。万维网文档所驻留的主机则运行服务器程序，因此这台主机也称为万维网服务器。**客户程序向服务器程序发出请求，服务器程序向客户程序送回客户所要的万维网文档。**在一个客户程序主窗口上显示出的万维网文档称为**页面**(page)。

从以上所述可以看出，万维网必须解决以下几个问题：

- (1) 怎样标志分布在整个互联网上的万维网文档？
- (2) 用什么样的协议来实现万维网上的各种链接？
- (3) 怎样使不同作者创作的不同风格的万维网文档，都能在互联网上的各种主机上显示出来，同时使用户清楚地知道在什么地方存在着链接？
- (4) 怎样使用户能够很方便地找到所需的信息？

为了解决第一个问题，万维网使用**统一资源定位符 URL (Uniform Resource Locator)**来标志万维网上的各种文档，并使每一个文档在整个互联网的范围内具有唯一的标识符 URL。为了解决上述的第二个问题，就要使万维网客户程序与万维网服务器程序之间的交互遵守严格的协议，这就是**超文本传送协议 HTTP (HyperText Transfer Protocol)**。HTTP 是一个应用层协议，它使用 TCP 连接进行可靠的传送。为了解决上述的第三个问题，万维网使用**超文本标记语言 HTML (HyperText Markup Language)**，使得万维网页面的设计者可以很方便地用链接从本页面的某处链接到互联网上的任何一个万维网页面，并且能够在自己的主机屏幕上将这些页面显示出来。最后，用户可使用搜索工具在万维网上方便地查找所需的信息。

下面我们将进一步讨论上述重要概念。

6.4.2 统一资源定位符 URL

1. URL 的格式

统一资源定位符 URL 是用来表示从互联网上得到的资源位置和访问这些资源的方法。URL 给资源的位置提供一种抽象的识别方法，并用这种方法给资源定位。只要能够对资源定位，系统就可以对资源进行各种操作，如存取、更新、替换和查找其属性。由此可见，URL 实际上就是在互联网上的资源的地址。只有知道了这个资源在互联网上的什么地方，才能对它进行操作。显然，互联网上的所有资源，都有一个唯一确定的 URL。

这里所说的“资源”是指在互联网上可以被访问的任何对象，包括文件目录、文件、文档、图像、声音等，以及与互联网相连的任何形式的数据。

URL 相当于一个文件名在网络范围的扩展。因此，URL 是与互联网相连的机器上的任何可访问对象的一个指针。由于访问不同对象所使用的协议不同，所以 URL 还指出读取某个对象时所使用的协议。URL 的一般形式由以下四个部分组成：

通常省略

主机名是万维网文档所存放的主机的域名，通常以 www 开头，但这并不是硬性规定。主机名用点分十进制的 IP 地址代替也是可以的。

最后的路径可能是较长的字符串（其中还可包括若干斜线/），但有时也不需要。在路径后面可能还有一些选项，这里不进行介绍了。

下面我们简单介绍使用得最多的一种 URL，即协议 HTTP。

使用协议 HTTP 的 URL 最常用的形式是把“:端口”省略:

若再将 URL 中的路径省略, 则 URL 就指明互联网上的某个主页(home page)。主页是个很重要的概念, 它可以是以下几种情况之一:

- 例如，要查有关清华大学的信息，就可先进入到清华大学的主页，其 URL 为^①：

这里省略了默认的端口号 80。我们从清华大学的主页入手,就可以通过许多不同的链接找到所要查找的各种有关清华大学各个部门的信息。

http://www.tsinghua.edu.cn/publish/newthu/newthu_cnt/faculties/index.html

主机域名
路径名

• 275 •

是清华大学的“院系设置”页面的 URL。注意：上面的 URL 中使用了指向文件的路径，而文件名就是最后的 index.htm。后缀 .htm（有时可写为 .html）表示这是一个用超文本标记语言 HTML 写出的文件。

URL 的“协议”和“主机名”部分，字母不区分大小写。但“路径”中的字符有时要区分大小写。

用户使用 URL 并非仅仅能够访问万维网的页面，而且还能够通过 URL 使用其他的互联网应用程序，如 FTP 或 USENET 新闻组等。更重要的是，用户在使用这些应用程序时，只使用一个程序，即浏览器。这显然是非常方便的。

6.4.3 超文本传送协议 HTTP

1. HTTP 的操作过程

协议 HTTP 定义了浏览器（即万维网客户进程）怎样向万维网服务器请求万维网文档，以及服务器怎样把文档传送给浏览器。从层次的角度看，HTTP 是面向事务的(transaction-oriented)^①应用层协议，它是万维网上能够可靠地交换文件（包括文本、声音、图像等各种多媒体文件）的重要基础。请注意，协议 HTTP 不仅传送完成超文本跳转所必需的信息，而且也传送任何可从互联网上得到的信息，如文本、超文本、声音和图像等。

万维网的大致工作过程如图 6-8 所示。

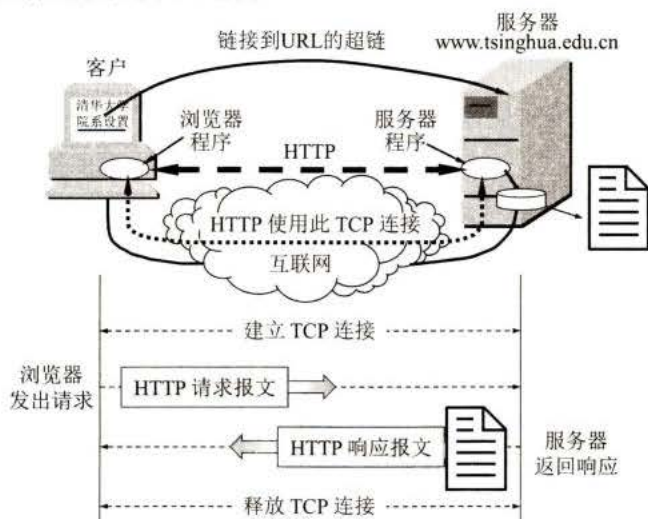


图 6-8 万维网的工作过程

每个万维网网点都有一个服务器进程，它不断地监听 TCP 的端口 80，以便发现是否有浏览器（即万维网客户。请注意，浏览器和万维网客户是同义词）向它发出连接建立请求。一旦监听到连接建立请求并建立了 TCP 连接之后，浏览器就向万维网服务器发出浏览某个页面的请求，服务器接着就返回所请求的页面作为响应。服务器在完成任务后，TCP 连接就被释放了。在浏览器和服务器的请求和响应的交互，必须按照规定的格式和遵循一定

^① 注：所谓事务(transaction)就是指一系列的信息交换，而这一系列的信息交换是一个不可分割的整体，也就是说，要么所有的信息交换都完成，要么一次交换都不进行。

的规则。这些格式和规则就是超文本传送协议 HTTP。

HTTP 规定在 HTTP 客户与 HTTP 服务器之间的每次交互，都由一个 ASCII 码串构成的请求和一个类似的通用互联网扩充，即“类 MIME (MIME-like)”的响应组成。HTTP 报文通常都使用 TCP 连接传送。

用户浏览页面的方法有两种。一种方法是在浏览器的地址窗口中键入所要找的页面的 URL。另一种方法是在某一个页面中用鼠标点击一个可选部分，这时浏览器会自动在互联网上找到所要链接的页面。

HTTP 使用了面向连接的 TCP 作为运输层协议，保证了数据的可靠传输。HTTP 不必考虑数据在传输过程中被丢弃后又怎样被重传。但是，协议 HTTP 本身是无连接的。这就是说，虽然 HTTP 使用了 TCP 连接，但通信的双方在交换 HTTP 报文之前不需要先建立 HTTP 连接。在 1997 年以前使用的是协议 HTTP/1.0 [RFC 1945]。现在普遍使用的升级版本是建议标准 HTTP/1.1 [RFC 7231]。2015 年以后，又有了新的建议标准 HTTP/2 [RFC 7540]，以及压缩 HTTP 报文首部的建议标准[RFC 7541]。

协议 HTTP 是无状态的(stateless)。也就是说，同一个客户第二次访问同一个服务器上的页面时，服务器的响应与第一次被访问时的相同（假定现在服务器还没有把该页面更新），因为服务器并不记得曾经访问过的这个客户，也不记得为该客户曾经服务过多少次。HTTP 的无状态特性简化了服务器的设计，使服务器更容易支持大量并发的 HTTP 请求。

下面我们粗略估算一下，从浏览器请求一个万维网文档到收到整个文档所需的时间（如图 6-9 所示）。用户在点击鼠标链接某个万维网文档时，协议 HTTP 首先要和服务器建立 TCP 连接。这需要三报文握手。当建立 TCP 连接的三报文握手的前两个报文完成后（即经过了一个 RTT 时间后），万维网客户就把 HTTP 请求报文，作为建立 TCP 连接的三报文握手中的第三个报文的数据，发送给万维网服务器。服务器收到 HTTP 请求报文后，就把所请求的文档作为响应报文返回给客户。

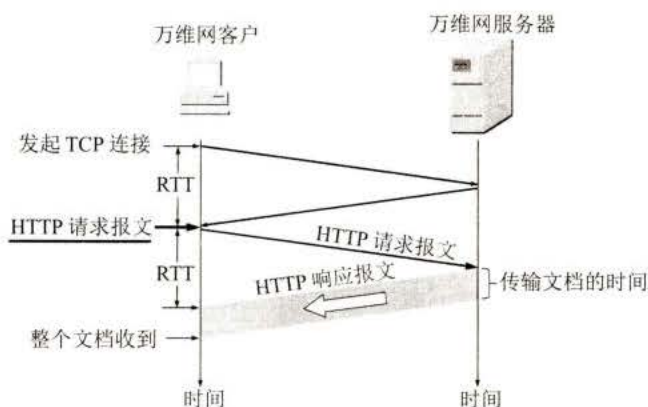


图 6-9 请求一个万维网文档所需的时间

从图 6-9 可看出，请求一个万维网文档所需的时间是该文档的传输时间（与文档大小成正比）加上两倍往返时间 RTT（一个 RTT 用于连接 TCP 连接，另一个 RTT 用于请求和接收万维网文档。TCP 建立连接的三报文握手的前两个报文段中的数据，就是客户对万维网文档的请求报文）。

协议 HTTP/1.0 的主要缺点，就是每请求一个文档就要有两倍 RTT 的开销。若一个主页

上有很多链接的对象（如图片等）需要依次进行链接，那么每一次链接下载都导致 $2 \times \text{RTT}$ 的开销。另一种开销就是万维网客户和服务端每一次建立新的 TCP 连接都要分配缓存和变量。特别是万维网服务器往往要同时服务于大量客户的请求，所以这种非持续连接会使万维网服务器的负担很重。好在浏览器都能够打开 5 ~ 10 个并行的 TCP 连接，而每一个 TCP 连接处理客户的一个请求。因此，使用并行 TCP 连接可以缩短响应时间。

协议 HTTP/1.1 较好地解决了这个问题，它使用了持续连接(persistent connection)。所谓持续连接就是万维网服务器在发送响应后仍然在一段时间内保持这条连接，使同一个客户（浏览器）和该服务器可以继续在这条连接上传送后续的 HTTP 请求报文和响应报文。这并不局限于传送同一个页面上链接的文档，而是只要这些文档都在同一个服务器上就行。协议 HTTP/1.1 的持续连接有两种工作方式，即非流水线方式(without pipelining)和流水线方式(with pipelining)。

非流水线方式的特点，是客户在收到前一个响应后才能发出下一个请求。因此，在 TCP 连接已建立后，客户每访问一次对象都要用去一个往返时间 RTT。这比非持续连接要用去两倍 RTT 的开销，节省了建立 TCP 连接所需的一个 RTT 时间。但非流水线方式还是有缺点的，因为服务器在发送完一个对象后，其 TCP 连接就处于空闲状态，浪费了服务器资源。

流水线方式的特点，是客户在收到 HTTP 的响应报文之前就能够接着发送新的请求报文。于是一个接一个的请求报文到达服务器后，服务器就可连续发回响应报文。因此，使用流水线方式时，客户访问所有的对象只需花费一个 RTT 时间。流水线工作方式使 TCP 连接中的空闲时间减少，提高了下载文档效率。

最初网页是以文本为主，但很快发展到使用大量的图片、音频和视频，并且对页面的实时性要求也越来越高（如视频聊天或直播），这样就使得协议 HTTP/1.1 已无法跟上互联网的发展了。于是谷歌公司在 2009 年开发了软件 SPDY，用来提高协议 HTTP/1.1 的工作效率。IETF 在此基础上与谷歌合作完成了协议 HTTP/2，而谷歌也停止了对 SPDY 的继续完善工作。

协议 HTTP/2 是协议 HTTP/1.1 的升级版本，其 HTTP 方法/状态码/语义等都没有改变，其主要特点如下：

(1) 我们知道，HTTP/1.1 具有流水线的工作方式。这就是在 TCP 连接建立后，客户可以连续向服务器发出许多个请求，而不必等到收到一个响应后再发送下一个请求。但服务器发回响应时必须按先后顺序排队，逐个地发送给客户。有时遇到某个响应迟迟不能发回，那么排在后面的一些响应就必须等待很长的时间。HTTP/2 把服务器发回的响应变成可以并行地发回（使用同一个 TCP 连接），这就大大缩短了服务器的响应时间。

(2) 使用 HTTP/1.1 时，当客户收到服务器发回的响应后，原来建立的 TCP 连接就释放了。如果客户还要继续向该服务器发送新的请求，就必须重新建立 TCP 连接。HTTP/2 允许客户复用 TCP 连接进行多个请求，这样就节省了 TCP 连续多次建立和释放连接所花费的时间。

(3) HTTP/1.1 的请求和响应报文是面向文本的(text-oriented)。当客户连续发送请求并受到响应时，在 TCP 连接上传送的 HTTP 报文首部成为不小的开销。在这些首部中有很多字段是重复的。为此，HTTP/2 把所有的报文都划分为许多较小的二进制编码的帧，并采用了新的压缩算法，不发送重复的首部字段，大大减小了首部的开销，提高了传输效率。

因此，现在主流的浏览器都支持 HTTP/2。但是，有的服务器还未来得及更新，仍然只

支持 HTTP/1.1。但 HTTP/2 是向后兼容的。当使用 HTTP/2 的客户向服务器发出请求时，如果服务器仍然使用 HTTP/1.1，那么服务器仍然可以收到请求报文。在发回响应后，客户就改用 HTTP/1.1 与服务器进行交互。

2. 代理服务器

代理服务器(proxy server)是一种网络实体，它又称为万维网高速缓存(Web cache)。代理服务器把最近的一些请求和响应暂存在本地磁盘中。当新请求到达时，若代理服务器发现这个请求与暂时存放的请求相同，就返回暂存的响应，而不需要按 URL 的地址再次去互联网访问该资源。代理服务器可在客户端或服务端工作，也可在中间系统上工作。下面我们例子说明它的作用。

设图 6-10(a)是校园网不使用代理服务器的情况。这时，校园网中所有的计算机都通过 2 Mbit/s 专线链路 (R_1-R_2) 与互联网上的源点服务器建立 TCP 连接。因而校园网各计算机访问互联网的通信量往往会使这条 2 Mbit/s 的链路过载，使得时延大大增加。

图 6-10(b)是校园网使用代理服务器的情况。这时，访问互联网的过程是这样的：

(1) 校园网的计算机中的浏览器向互联网的服务器请求服务时，就先和校园网的代理服务器建立 TCP 连接，并向代理服务器发出 HTTP 请求报文（见图 6-10(b)中的①）。

(2) 若代理服务器已经存放了所请求的对象，代理服务器就把这个对象放入 HTTP 响应报文中返回给计算机的浏览器。

(3) 否则，代理服务器就代表发出请求的用户浏览器，与互联网上的源点服务器(origin server)建立 TCP 连接（如图 6-10(b)中的②所示），并发送 HTTP 请求报文。

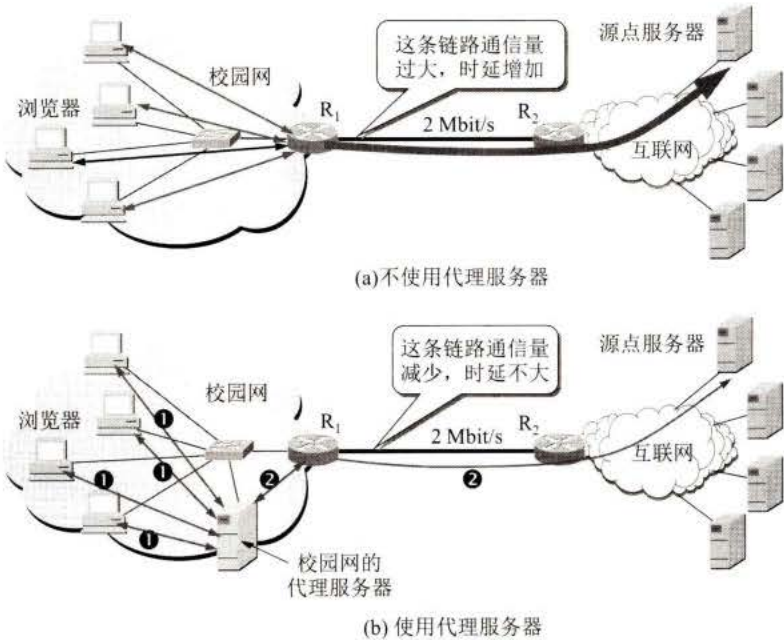


图 6-10 代理服务器的作用

(4) 源点服务器把所请求的对象放在 HTTP 响应报文中返回给校园网的代理服务器。

(5) 代理服务器收到这个对象后，先复制在自己的本地存储器中（留待以后用），然后再把这个对象放在 HTTP 响应报文中，通过已建立的 TCP 连接（见图 6-10(b)中的①），返回

给请求该对象的浏览器。

我们注意到，代理服务器有时是作为服务器（当接受浏览器的 HTTP 请求时），但有时却作为客户（当向互联网上的源点服务器发送 HTTP 请求时）。

在使用代理服务器的情况下，由于有相当大一部分通信量局限在校园网的内部，因此，2 Mbit/s 专线链路（ R_1-R_2 ）上的通信量大大减少，因而减小了访问互联网的时延。

以代理服务器方式构成的内容分发网络 CDN (Content Distribution Network)在互联网应用中起到了很大的作用。最先使用 CDN 技术的是美国的 Akamai (阿卡迈)公司。Akamai 公司现在是全球最大的 CDN 平台，是全球能够提供最大规模分发在线视频的服务商。目前 Akamai 公司拥有 24 万台服务器，3900 个位置节点，使全球的用户可以就近下载视频、音频节目，或进行其他业务联系，而用户甚至根本不知道已经使用了 Akamai 提供的服务。现在 Akamai 与全球多家电信运营商建立了深度合作关系，所提供的通信流量高达每秒 40 TB，业务遍及 137 个国家和地区。

扫一扫

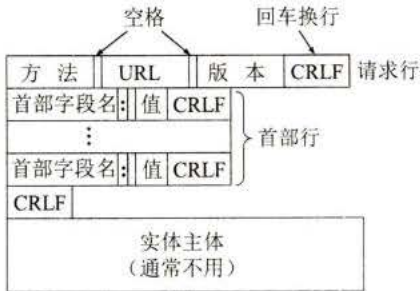


视频讲解

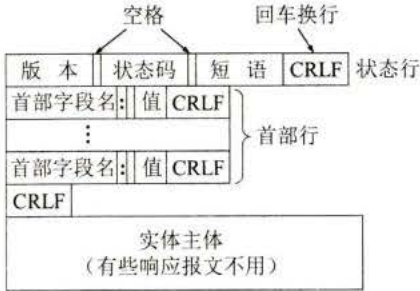
3. HTTP 的报文结构

HTTP 有两类报文：

- (1) 请求报文——从客户向服务器发送请求报文，如图 6-11(a)所示。
- (2) 响应报文——从服务器到客户的回答，如图 6-11(b)所示。



(a) 请求报文



(b) 响应报文

图 6-11 HTTP 的报文结构

由于 HTTP 是面向文本的，因此在报文中的每一个字段都是一些 ASCII 码串，因而各个字段的长度都是不确定的。

HTTP 请求报文和响应报文都是由三个部分组成的。可以看出，这两种报文格式的区别就是开始行不同。

(1) 开始行，用于区分是请求报文还是响应报文。在请求报文中的开始行叫作请求行 (Request-Line)，而在响应报文中的开始行叫作状态行 (Status-Line)。在开始行的三个字段之间都以空格分隔开，最后的“CR”和“LF”分别代表“回车”和“换行”。

(2) 首部行，用来说明浏览器、服务器或报文主体的一些信息。首部可以有好几行，但也可以不使用。在每一个首部行中都有首部字段名和它的值，每一行在结束的地方都要有“回车”和“换行”。整个首部行结束时，还有一空行将首部行和后面的实体主体分开。

(3) 实体主体 (entity body)，在请求报文中一般都不用这个字段，而在响应报文中也可能没有这个字段。

下面先介绍 HTTP 请求报文的一些主要特点。

请求报文的第一行“请求行”只有三个内容，即方法、请求资源的 URL，以及 HTTP 的版本。

请注意：这里的名词“方法”(method)是面向对象技术中使用的专门名词。所谓“方法”就是对所请求的对象进行的操作，这些方法实际上也就是是一些命令。因此，请求报文的类型是由它所采用的方法决定的。表 6-1 给出了请求报文中常用的几种方法。

表 6-1 HTTP 请求报文的一些方法

方法（操作）	意义
OPTION	请求一些选项的信息
GET	请求读取由 URL 所标志的信息
HEAD	请求读取由 URL 所标志的信息的首部
POST	给服务器添加信息（例如，注释）
PUT	在指明的 URL 下存储一个文档
DELETE	删除指明的 URL 所标志的资源
TRACE	用来进行环回测试的请求报文
CONNECT	用于代理服务器

下面是 HTTP 的请求报文的开始行（即请求行）的格式。请注意，在 GET 后面有一个空格，接着是某个完整的 URL，其后面又有一个空格，最后是 HTTP/1.1。

```
GET http://www.xyz.edu.cn/dir/index.htm HTTP/1.1
```

下面是一个完整的 HTTP 请求报文的例子：

```
GET /dir/index.htm HTTP/1.1           {请求行使用了相对 URL}
Host: www.xyz.edu.cn                  {此行是首部行的开始。这行给出主机的域名}
Connection: close                      {告诉服务器发送完请求的文档后就可释放连接}
User-Agent: Mozilla/5.0                {表明用户代理是使用火狐浏览器 Firefox}
Accept-Language: cn                    {表示用户希望优先得到中文版本的文档}
                                       {请求报文的最后还有一个空行}
```

在请求行使用了相对 URL（即省略了主机的域名）是因为下面的首部行（第 2 行）给出了主机的域名。第 3 行是告诉服务器不使用持续连接，表示浏览器希望服务器在传送完所请求的对象后即关闭 TCP 连接。这个请求报文没有实体主体。

再看一下 HTTP 响应报文的主要特点。

每一个请求报文发出后，都能收到一个响应报文。响应报文的第一行就是状态行。

状态行包括三项内容，即 HTTP 的版本、状态码，以及解释状态码的简单短语。

状态码(Status-Code)都是三位数字的，分为 5 大类，原先有 33 种[RFC 2616]，后来又增加了几种[RFC 6585，建议标准]。这 5 大类的状态码都是以不同的数字开头的。

1xx 表示通知信息，如请求收到了或正在进行处理。

2xx 表示成功，如接受或知道了。

3xx 表示重定向，如要完成请求还必须采取进一步的行动。

4xx 表示客户的差错，如请求中有错误的语法或不能完成。

5xx 表示服务器的差错，如服务器失效无法完成请求。

下面三种状态行在响应报文中是经常见到的。

HTTP/1.1 202 Accepted	{接受}
HTTP/1.1 400 Bad Request	{错误的请求}
Http/1.1 404 Not Found	{找不到}

若请求的网页从 <http://www.ee.xyz.edu/index.html> 转移到了一个新的地址，则响应报文的状态行和一个首部行就是下面的形式：

HTTP/1.1 301 Moved Permanently	{永久性地转移了}
Location: http://www.xyz.edu/ee/index.html	{新的 URL}

4. 在服务器上存放用户的信息

在本节（6.4.3 节）第 1 小节“HTTP 的操作过程”中已经讲过，HTTP 是无状态的。这样做虽然简化了服务器的设计，但在实际工作中，一些万维网站点却常常希望能够识别用户。例如，在网上购物时，一个顾客要购买多种物品。当他把选好的一件物品放入“购物车”后，他还要继续浏览和选购其他物品。因此，服务器需要记住用户的身份，使他接着选购的一些物品能够放入同一个“购物车”中，这样就便于集中结账。有时某些万维网站点也可能想限制某些用户的访问。要做到这点，可以在 HTTP 中使用 Cookie。在 RFC 6265 中对 Cookie 进行了定义，规定万维网站点可以使用 Cookie 来跟踪用户。Cookie 原意是“小甜饼”（广东人用方言音译为“曲奇”），目前尚无标准译名，在这里 Cookie 表示在 HTTP 服务器和客户之间传递的状态信息。现在很多网站都已广泛使用 Cookie。

Cookie 是这样工作的：当用户 A 浏览某个使用 Cookie 的网站时，该网站的服务器就为 A 产生一个唯一的识别码，并以此作为索引在服务器的后端数据库中产生一个项目。接着在给 A 的 HTTP 响应报文中添加一个叫作 Set-cookie 的首部行。这里的“首部字段名”就是“Set-cookie”，而后面的“值”就是赋予该用户的“识别码”。例如这个首部行是这样的：

```
Set-cookie: 31d4d96e407aad42
```

当 A 收到这个响应时，其浏览器就在它管理的特定 Cookie 文件中添加一行，其中包括这个服务器的主机名和 Set-cookie 后面给出的识别码。当 A 继续浏览这个网站时，每发送一个 HTTP 请求报文，其浏览器就会从其 Cookie 文件中取出这个网站的识别码，并放到 HTTP 请求报文的 Cookie 首部行中：

```
Cookie: 31d4d96e407aad42
```

于是，这个网站就能够跟踪用户 31d4d96e407aad42（用户 A）在该网站的活动。需要注意的是，服务器并不需要知道这个用户的真实姓名以及其他的信息。但服务器能够知道用户 31d4d96e407aad42 在什么时间访问了哪些页面，以及访问这些页面的顺序。如果 A 是在网上购物，那么这个服务器可以为 A 维护一个所购物品的列表，使 A 在结束这次购物时可以一起付费。

如果 A 在几天后再次访问这个网站，那么他的浏览器会在其 HTTP 请求报文中继续使用首部行 Cookie: 31d4d96e407aad42，而这个网站服务器根据 A 过去的访问记录可以向他推荐商品。如果 A 已经在该网站登记过和使用过信用卡付费，那么这个网站就已经保

存了 A 的姓名、电子邮件地址、信用卡号码等信息。这样，当 A 继续在该网站购物时，只要还使用同一个计算机上网，由于浏览器产生的 HTTP 请求报文中都携带了同样的 Cookie 首部行，服务器就可利用 Cookie 来验证出这是用户 A，因此以后 A 在这个网站购物时就不必重新在键盘上输入姓名、信用卡号码等信息。这对顾客显然是很方便的。

尽管 Cookie 能够简化用户网上购物的过程，但 Cookie 的使用一直引起很多争议。有人认为 Cookie 会把计算机病毒带到用户的计算机中。其实这是对 Cookie 的误解。Cookie 只是一个小小的文本文件，不是计算机的可执行程序，因此不可能传播计算机病毒，也不可能用来获取用户计算机硬盘中的信息。对于 Cookie 的另一个争议，是关于用户隐私的保护问题。例如，网站服务器知道了 A 的一些信息，就有可能把这些信息出卖给第三方。Cookie 还可用来收集用户在万维网网站上的行为。这些都属于用户个人的隐私。有些网站为了使顾客放心，就公开声明他们会保护顾客的隐私，绝对不会把顾客的识别码或个人信息出售或转移给其他厂商。

为了让用户有拒绝接受 Cookie 的自由，在浏览器中用户可自行设置接受 Cookie 的条件。例如在浏览器 IE 11.0 中，选择工具栏中的“工具”→“Internet 选项”→“隐私”命令，就可以看见菜单中的左边有一个可上下滑动的标尺，它有六个位置。最高的位置是阻止所有 Cookie，而最低的位置是接受所有 Cookie。中间的位置则是在不同条件下可以接受 Cookie。用户可根据自己的情况对 IE 浏览器进行必要的设置。

6.4.4 万维网的文档

1. 超文本标记语言 HTML

要使任何一台计算机都能显示出任何一个万维网服务器上的页面，就必须解决页面制作的标准化问题。超文本标记语言 HTML (HyperText Markup Language)就是一种制作万维网页面的标准语言，它消除了不同计算机之间信息交流的障碍。但请注意，HTML 并不是应用层的协议，它只是万维网浏览器使用的一种语言。由于 HTML 非常易于掌握且实施简单，因此它很快就成为万维网的重要基础[RFC 2854]。官方的 HTML 标准由万维网联盟 W3C（即 WWW Consortium）负责制定。有关 HTML 的一些参考资料见[W-HTML]。从 HTML 在 1993 年问世后，就不断地对其版本进行更新。现在最新的版本是 HTML 5.0，新的版本增加了<audio>和<video>两个标签来实现对多媒体中的音频、视频使用的支持，增加了能够在移动设备上支持多媒体功能。

HTML 定义了许多用于排版的命令，即“标签”(tag)^①。例如，<I>表示后面开始用斜体字排版，而</I>则表示斜体字排版到此结束。HTML 把各种标签嵌入到万维网的页面中，这样就构成了所谓的 HTML 文档。HTML 文档是一种可以用任何文本编辑器（例如，Windows 的记事本 Notepad）创建的 ASCII 码文件。但应注意，仅当 HTML 文档是以.html 或.htm 为后缀时，浏览器才对这样的 HTML 文档的各种标签进行解释。如果 HTML 文档改为以.txt 为其后缀，则 HTML 解释程序就不对标签进行解释，而浏览器只能看见原来的文本文件。

^① 注：在[MINGCI93]中，将 tag 和 flag 两个名词都译为“标志”。由于目前已有较多的作者将 tag 译为“标签”，并考虑到最好与 flag 的译名有所区别，故将 tag 译为**标签**。实际上，“标签”的意思也还比较准确，因为一个 HTML 文档与浏览器所显示的内容相比，主要就是增加了许多的标签。

并非所有的浏览器都支持所有的 HTML 标签。若某一个浏览器不支持某一个 HTML 标签，则浏览器将忽略此标签，但在一对不能识别的标签之间的文本仍然会被显示出来。

下面是一个简单例子，用来说明 HTML 文档中标签的用法。在每一个语句后面的花括号中的字是给读者看的注释，在实际的 HTML 文档中并没有这种注释。

<HTML>	{HTML 文档开始}
<HEAD>	{首部开始}
<TITLE>一个 HTML 的例子</TITLE>	{“一个 HTML 的例子”是文档的标题}
</HEAD>	{首部结束}
<BODY>	{主体开始}
<H1>HTML 很容易掌握</H1>	{“HTML 很容易掌握”是主体的 1 级题头}
<P>这是第一个段落。</P>	{<P>和</P>之间的文字是一个段落}
<P>这是第二个段落。</P>	{<P>和</P>之间的文字是一个段落}
</BODY>	{主体结束}
</HTML>	{HTML 文档结束}

把上面的 HTML 文档存入 D 盘的文件夹 HTML 中，文件名是 HTML-example.html（注意：实际的文档中没有注释部分）。当浏览器读取了该文档后，就按照 HTML 文档中的各种标签，根据浏览器所使用的显示器的尺寸和分辨率大小，重新进行排版并显示出来。图 6-12 表示 IE 浏览器在计算机屏幕上显示出的与该文档有关部分的画面。文档的标题(title)“一个 HTML 的例子”显示在浏览器最上面的标题栏中。文件的路径显示在地址栏中。再下面就是文档的主体部分。主体部分的题头(heading)，即文档主体部分的标题“HTML 很容易掌握”，用较大的字号显示出来，因为在标签中指明了使用的是 1 级题头<H1>。

目前已开发出了很好的制作万维网页面的软件工具，使我们能够像使用 Word 文字处理器那样很方便地制作各种页面。即使我们用 Word 文字处理器编辑了一个文件，但只要在“另存为(Save As)”时选取文件后缀为.htm 或.html，就可以很方便地把 Word 的.doc 格式文件转换为浏览器可以显示的 HTML 格式的文档。



图 6-12 在屏幕上显示的 HTML 文档主体部分的例子

HTML 允许在万维网页面中插入图像。一个页面本身带有的图像称为**内含图像**(inline image)。HTML 标准并没有规定该图像的格式。实际上，大多数浏览器都支持 GIF 和 JPEG 文件。很多格式的图像占据的存储空间太大，因而这种图像在互联网传送时就很浪费时间。例如，一幅**位图文件**(.bmp)可能要占用 500 ~ 700 KB 的存储空间。但若将此图像改存为经压缩的 .gif 格式，则可能只有十几个千字节，大大减少了存储空间。

HTML 还规定了链接的设置方法。我们知道每个链接都有一个**起点和终点**。链接的起点说明在万维网页面中的什么地方可引出一个链接。在一个页面中，链接的起点可以是一个字或几个字，或是一幅图，或是一段文字。在浏览器所显示的页面上，链接的起点是很容易

识别的。在以文字作为链接的起点时，这些文字往往用不同的颜色显示（例如，一般的文字用黑色字时，链接起点往往使用蓝色字），甚至还会加上下划线（一般由浏览器来设置）。当我们将鼠标移动到一个链接的起点时，表示鼠标位置的箭头就变成了一只手。这时只要点击鼠标，这个链接就被激活。

链接的终点可以是其他网站上的页面。这种链接方式叫作**远程链接**。这时必须在 HTML 文档中指明链接到的网站的 URL。有时链接可以指向本计算机中的某一个文件或本文件中的某处，这叫作**本地链接**。这时必须在 HTML 文档中指明链接的路径。

实际上，现在这种链接方式已经不局限于用在万维网文档中。在最常用的 Word 文字处理器的工具栏中，也设有“插入超链接”的按钮。只要点击这个按钮，就可以看到设置超链接的窗口。用户可以很方便地在自己写的 Word 文档中设置各种链接的起点和终点。

在这一小节的最后，我们还要简单介绍一下和浏览器有关的几种其他语言。

XML (Extensible Markup Language)是**可扩展标记语言**，它和 HTML 很相似。但 XML 的设计宗旨是传输数据，而不是显示数据（HTML 是为了在浏览器上显示数据）。更具体些，XML 用于标记电子文件，使其具有结构性的标记语言，可用来标记数据、定义数据类型，是一种允许用户对自己的标记语言进行定义的源语言。XML 是一种简单、与平台无关并被广泛采用的标准。XML 相对于 HTML 的优点是它将用户界面与结构化数据分隔开来。这种数据与显示的分离使得集成来自不同源的数据成为可能。客户信息、订单、研究结果、账单付款、病历、目录数据及其他信息都可以转换为 XML。XML 不是要替换 HTML，而是对 HTML 的补充。XML 标记由文档的作者定义，并且是无限制的。HTML 标记则是预定义的；HTML 作者只能使用当前 HTML 标准所支持的标记。

另一种语言 XHTML (Extensible HTML)是**可扩展超文本标记语言**，它与 HTML 4.01 几乎是相同的。但 XHTML 是更严格的 HTML 版本，也是一个 W3C 标准（2000 年 1 月制定），是作为一种 XML 应用被重新定义的 HTML，并将逐渐取代 HTML。所有新的浏览器都支持 XHTML。

还有一种语言 CSS (Cascading Style Sheets)是**层叠样式表**，它是一种样式表语言，用于为 HTML 文档定义布局。CSS 与 HTML 的区别就是：HTML 用于结构化内容，而 CSS 则用于格式化结构化的内容。例如，在浏览器上显示的字体、颜色、边距、高度、宽度、背景图像等方面，都能够给出精确的规定。现在所有的浏览器都支持 CSS。

2. 动态万维网文档

上面所讨论的万维网文档只是万维网文档中最基本的一种，即所谓的**静态文档**(static document)。静态文档在文档创作完毕后就存放在万维网服务器中，在被用户浏览的过程中，内容不会改变。由于这种文档的内容不会改变，因此用户对静态文档的每次读取所得到的返回结果都是相同的。

静态文档的最大优点是简单。由于 HTML 是一种排版语言，因此静态文档可以由不懂程序设计的人员来创建。但静态文档的缺点是不够灵活。当信息变化时就要由文档的作者手工对文档进行修改。可见，变化频繁的文档不适于做成静态文档。

动态文档(dynamic document)是指文档的内容是在浏览器访问万维网服务器时才由应用程序动态创建的。当浏览器请求到达时，万维网服务器要运行另一个应用程序，并把控制转移到此应用程序。接着，该应用程序对浏览器发来的数据进行处理，并输出 HTML 格式的文

档，万维网服务器把应用程序的输出作为对浏览器的响应。由于对浏览器每次请求的响应都是临时生成的，因此用户通过动态文档所看到的内容是不断变化的。动态文档的主要优点是具有报告当前最新信息的能力。例如，动态文档可用来报告股市行情、天气预报或民航售票情况等内容。但动态文档的创建难度比静态文档的高，因为动态文档的开发不是直接编写文档本身，而是编写用于生成文档的应用程序，这就要求动态文档的开发人员必须会编程，而所编写的程序还要通过大范围的测试，以保证输入的有效性。

动态文档和静态文档之间的主要差别体现在服务器一端。这主要是文档内容的生成方法不同。而从浏览器的角度看，这两种文档并没有区别。动态文档和静态文档的内容都遵循 HTML 所规定的格式，浏览器仅根据在屏幕上看到的内容无法判定服务器送来的是哪一种文档，只有文档的开发者才知道。

从以上所述可以看出，要实现动态文档就必须在以下两个方面对万维网服务器的功能进行扩充：

(1) 应增加另一个应用程序，用来处理浏览器发来的数据，并创建动态文档。

(2) 应增加一个机制，用来使万维网服务器将浏览器发来的数据传送给这个应用程序，然后万维网服务器能够解释这个应用程序的输出，并向浏览器返回 HTML 文档。

图 6-13 是扩充了功能的万维网服务器的示意图。这里增加了一个机制，叫作通用网关接口 CGI (Common Gateway Interface)。CGI 是一种标准，它定义了动态文档应如何创建，输入数据应如何提供给应用程序，以及输出结果应如何使用。

在万维网服务器中新增加的应用程序叫作 CGI 程序。取这个名字的原因是：万维网服务器与 CGI 的通信遵循 CGI 标准。“通用”是因为这个标准所定义的规则对其他任何语言都是通用的。“网关”二字的出现是因为 CGI 程序还可能访问其他的服务器资源，如数据库或图形软件包，因而 CGI 程序的作用有点像一个网关。也有人将 CGI 程序简称为网关程序。“接口”是因为有一些已定义好的变量和调用等可供其他 CGI 程序使用。请读者注意：在看到 CGI 这个名词时，应弄清是指 CGI 标准，还是指 CGI 程序。

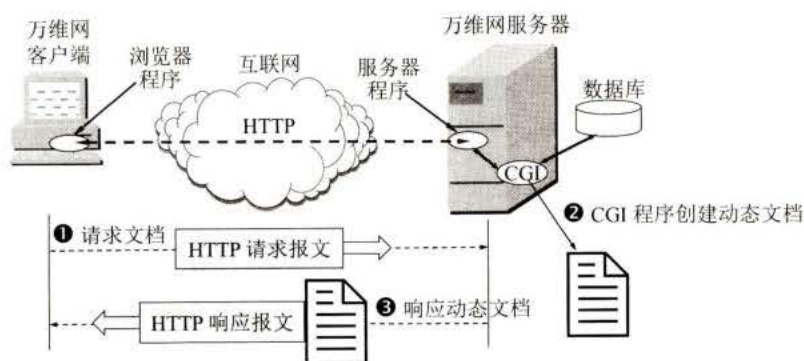


图 6-13 扩充了功能的万维网服务器

CGI 程序的正式名字是 CGI 脚本(script)。按照计算机科学的一般概念，“脚本”^①指的

^① 注：脚本(script)一词还有其他的意思。例如，在多媒体开发程序中用“脚本”来表示编程人员输入的一系列指令，这些指令指明多媒体文件应按什么顺序执行。

是一个程序，它被另一个程序（解释程序）而不是计算机的处理机来解释或执行。有一些语言专门作为脚本语言(script language)，如 Perl, REXX（在 IBM 主机上使用），JavaScript 以及 Tcl/Tk 等。脚本也可用一些常用的编程语言写出，如 C, C++等。使用脚本语言可更容易和更快地进行编码，这对一些有限功能的小程序是很合适的。但一个脚本运行起来比一般的编译程序要慢，因为它的每一条指令先要被另一个程序来处理（这就要一些附加的指令），而不是直接被指令处理器来处理。脚本不一定是一个独立的程序，它可以是一个动态装入的库，甚至是服务器的一个子程序。

CGI 程序又称为 cgi-bin 脚本，这是因为在许多万维网服务器上，为便于找到 CGI 程序，就将 CGI 程序放在/cgi-bin 的目录下。

3. 活动万维网文档

随着 HTTP 和万维网浏览器的发展，上一节所述的动态文档已明显地不能满足发展的需要。这是因为，动态文档一旦建立，它所包含的信息内容也就固定下来而无法及时刷新屏幕。另外，像动画之类的显示效果，动态文档也无法提供。

有两种技术可用于浏览器屏幕显示的连续更新。一种技术称为服务器推送(server push)，这种技术是将所有的工作都交给服务器。服务器不断地运行与动态文档相关联的应用程序，定期更新信息，并发送更新过的文档。

尽管从用户的角度看，这样做可达到连续更新的目的，但这也有很大的缺点。首先，为了满足很多客户的请求，服务器就要运行很多服务器推送程序。这将造成过多的服务器开销。其次，服务器推送技术要求服务器为每一个浏览器客户维持一个不释放的 TCP 连接。随着 TCP 连接的数目增加，每一个连接所能分配到的网络带宽就下降，这就导致网络传输时延的增大。

另一种提供屏幕连续更新的技术是活动文档(active document)。这种技术是把所有的工作都转移给浏览器端。每当浏览器请求一个活动文档时，服务器就返回一段活动文档程序副本，使该程序副本在浏览器端运行。这时，活动文档程序可与用户直接交互，并可连续地改变屏幕的显示。只要用户运行活动文档程序，活动文档的内容就可以连续地改变。由于活动文档技术不需要服务器的连续更新传送，对网络带宽的要求也不会太高。

从传送的角度看，浏览器和服务器都把活动文档看成是静态文档。在服务器上的活动文档的内容是不变的，这点和动态文档是不同的。浏览器可在本地缓存一份活动文档的副本。活动文档还可处理成压缩形式，以便于存储和传送。另一点要注意的是，活动文档本身并不包括其运行所需的全部软件，大部分的支持软件是事先存放在浏览器中的。图 6-14 说明了活动文档的创建过程。

由美国 SUN 公司开发的 Java 语言是一项用于创建和运行活动文档的技术。在 Java 技术中使用了一个新的名词“小应用程序”(applet)^①来描述活动文档程序。当用户从万维网服务器下载一个嵌入了 Java 小应用程序的 HTML 文档后，用户可在浏览器的显示屏幕上点击某个图像，然后就可看到动画的效果；或是在某个下拉式菜单中点击某个项目，即可看到根

^① 注：在 Java 语言出现之前就已经有了 applet 这一名词。小应用程序 applet 通常被嵌入在操作系统或一个较大的应用程序之中。在万维网技术中，此名词常常指的是 Java 小应用程序。

据用户键入的数据所得到的计算结果。实际上, Java 技术是活动文档技术的一部分。限于篇幅, 有关 Java 技术的进一步讨论这里从略。

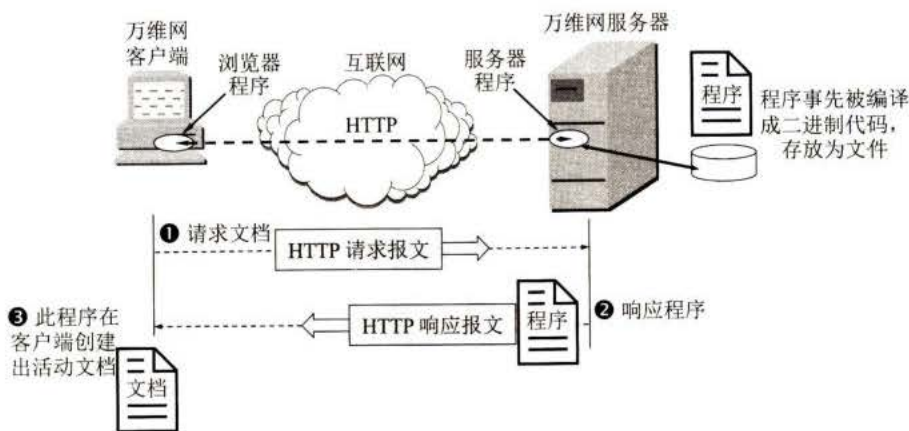


图 6-14 活动文档由服务器发送过来的程序在客户端创建

6.4.5 万维网的信息检索系统

1. 全文检索搜索与分类目录搜索

万维网是一个大规模的、联机式的信息储藏所。那么, 应当采用什么方法才能找到所需的信息呢? 如果已经知道存放该信息的网点, 那么只要在浏览器的地址(Location)框内键入该网点的 URL 并按回车键, 就可进入该网点。但是, 若不知道要找的信息在何网点, 那就要使用万维网的搜索工具。

在万维网中用来进行搜索的工具叫作**搜索引擎(search engine)**。搜索引擎的种类很多, 但大体上可划分为两大类, 即**全文检索搜索引擎**和**分类目录搜索引擎**。

全文检索搜索引擎是一种纯技术型的检索工具。它的工作原理是通过搜索软件(例如一种叫作“蜘蛛”或“网络机器人”的 Spider 程序)到互联网上的各网站收集信息, 找到一个网站后可以从这个网站再链接到另一个网站, 像蜘蛛爬行一样。然后按照一定的规则建立一个很大的在线索引数据库供用户查询。用户在查询时只要输入关键词, 就从**已经建立的索引数据库**里进行查询(并不是实时地在互联网上检索到的信息)。因此很可能有些查到的信息已经是过时的(例如很多年前的)。建立这种索引数据库的网站必须定期对已建立的数据库进行更新维护(但不少网站的维护很不及时, 因此对查找到的信息一定要注意其发布的时间)。现在全球最大的并且最受欢迎的全文检索搜索引擎就是谷歌 Google (www.google.com)。谷歌提供的主要的搜索服务有: 网页搜索、图片搜索、视频搜索、地图搜索、新闻搜索、购物搜索、博客搜索、论坛搜索、学术搜索、财经搜索等。应全球用户的需求, 谷歌在美国及世界各地创建数据中心。至 2013 年底, 谷歌的数据中心在全球共设有 12 处。大多数数据中心的业主基于信息安全考虑, 极少透露其数据中心的信息及内部情形。

我们将在下一小节简单介绍谷歌搜索技术的特点。现在“谷歌”不仅是网站名, 而且还是动词。例如, “谷歌一下”的意思就是“用谷歌网站进行信息搜索”。在全文检索搜索引擎中另外两个著名的网站是美国微软的必应(cn.bing.com)和中国的百度(www.baidu.com)。

分类目录搜索引擎并不采集网站的任何信息, 而是利用各网站向搜索引擎提交网站信

息时填写的关键词和网站描述等信息,经过人工审核编辑后,如果认为符合网站登录的条件,则输入到分类目录的数据库中,供网上用户查询。因此,分类目录搜索也叫作分类网站搜索。分类目录的好处就是用户可根据网站设计好的目录有针对性地逐级查询所需要的信息,查询时不需要使用关键词,只需要按照分类(先找大类,再找下面的小类),因而查询的准确性较好。但分类目录查询的结果并不是具体的页面,而是被收录网站主页的 URL 地址,因而所得到的内容就比较有限。相比之下,全文检索可以检索出大量的信息(一次检索的结果是几百万条,甚至是千万条以上),但缺点是查询结果不够准确,往往是罗列出了海量的信息(如上千万个页面),使用户无法迅速找到所需的信息。在分类目录搜索引擎中最著名的就是雅虎(www.yahoo.com)。国内著名的分类搜索引擎有雅虎中国(cn.yahoo.com)、新浪(sina.com.cn)、搜狐(www.sohu.com)、网易(www.163.com)等。

图 6-16 说明了上述这两种搜索方法的区别。图 6-15(a)是全文搜索谷歌的首页。用户只需在空白的栏目中键入拟搜索的关键词,搜索引擎就返回搜索结果,用户可根据屏幕上显示的结果继续点击下去,直到看到满意的结果。图 6-15(b)是分类检索新浪网的首页。我们可以看到页面上有三行共 63 个类别。用户要检索的内容通常总是在这几十个类别之中,因此按类别点击查找下去,最后就可以查找到所要检索的内容。



(a) 全文检索举例



(b) 分类检索举例

图 6-15 举例说明两种检索的区别

从用户的角度看,使用这两种不同的搜索引擎一般都能够实现自己查询信息的目的。为了使用户能够更加方便地搜索到有用信息,目前许多网站往往同时具有全文检索搜索和分类目录搜索的功能。在互联网上搜索信息需要经验的积累。要多实践才能掌握从互联网获取信息的技巧。

这里再强调一下,不管哪种搜索引擎,就是告诉你只要链接到什么地方就可以检索到所需的信息。搜索引擎网站本身并没有直接存储这些信息。

值得注意的是,目前出现了**垂直搜索引擎(vertical search engine)**,它针对某一特定领域、特定人群或某一特定需求提供搜索服务。垂直搜索也是提供关键字来进行搜索的,但被放到了一个行业知识的上下文中,返回的结果更倾向于信息、消息、条目等。例如,对买房的人讲,他希望查找的是房子的具体供求信息(如面积、地点、价格等),而不是有关房子供求的一般性的论文或新闻、政策等。目前热门的垂直搜索行业有:购物、旅游、汽车、求职、房产、交友等。还有一种**元搜索引擎(meta search engine)**,它把用户提交的检索请求发送到多个独立的搜索引擎上去搜索,并把检索结果集中统一处理,以统一的格式提供给用户,因此是搜索引擎之上的搜索引擎。它的主要精力放在提高搜索速度、智能化处理搜索结果、个

性化搜索功能的设置和用户检索界面的友好性上。元搜索引擎的查全率和查准率都比较高。

2. Google 搜索技术的特点

Google 的搜索引擎性能优良，因为它使用了先进的硬件和软件。以往的大多数的搜索引擎是使用少量大型服务器，在访问高峰期，搜索的速度就会明显减慢。Google 则利用在互联网上相互链接的计算机来快速查找每个搜索的答案，并且成功地缩短了查找的相应时间。Google 的搜索软件可同时进行许多运算，它的核心技术就是 PageRankTM，译为网页排名。

PageRank 对搜索出来的结果按重要性进行排序，这是 Google 的两个创始人 Larry Page 和 Sergey Brin 共同开发出来的[W-PageRank]。由于用户在有限的时间内，不可能阅读全部的搜索结果（因为数量往往非常大），而通常仅仅是查阅一下前几个（或前几十个）项目。因此用户希望检索结果能够按重要性来排序。但怎样确定某个页面的重要性呢？传统的搜索引擎往往是检查关键字在网页上出现的频率。PageRank 技术则把整个互联网当作了一个整体对待，检查整个网络链接的结构，并确定哪些网页重要性最高。更具体些，就是如果有很多网站上的链接都指向页面 A，那么页面 A 就比较重要。PageRank 对链接的数目进行加权统计。对来自重要网站的链接，其权重也较大。统计链接数目的问题是一个二维矩阵相乘的问题，从理论上讲，这种二维矩阵的元素数是网页数目的平方。对于 1 亿个网页，这个矩阵就有 1 亿亿个元素。这样大的矩阵相乘，计算量是非常大的。Larry Page 和 Sergey Brin 两人利用稀疏矩阵计算的技巧，大大地简化了计算量。他们用迭代的方法解决了这个问题。他们先假定所有网页的排名是相同的，并且根据此初始值，算出各个网页的第一次迭代排名，再根据第一次迭代排名算出第二次的排名。他们从理论上证明了不论初始值如何选取，这种算法都保证了网页排名的估计值能收敛到排名的真实值。这种算法是完全没有任何人工干预的，厂商不可能用金钱购买网页的排名。Google 还要进行超文本匹配分析，以确定哪些网页与正在执行的特定搜索相关。在综合考虑整体重要性以及与特定查询的相关性之后，Google 就把最相关、最可靠的搜索结果放在首位。

然而有一些著名网站通过“竞价排名”把虚假广告信息放在检索结果的首位，结果误导了消费者，使受骗者蒙受很大的损失。因此对网络搜索的结果，我们应认真分析其真伪，提高辨别能力，不要随意轻信网络检索的广告信息（哪怕是知名度很高的网站）。

6.4.6 博客和微博

近年来，万维网的一些新的应用广为流行，这就是博客和微博。下面进行简单的介绍。

1. 博客

我们知道，建立网站就是万维网的一种应用。博客(blog)和网站有很相似的地方。博客的作者可以源源不断地往万维网上的个人博客里填充内容，供其他网民阅读。网民可以用浏览器上网阅读博客、发表评论，也可以什么都不做。

博客是万维网日志(weblog)的简称。也有人把 blog 进行音译，译为“部落格”，或“部落阁”。还有人用“博文”来表示“博客文章”。

本来，网络日志是指个人撰写并在互联网上发布的、属于网络共享的个人日记。但现在它不仅可以是个人日记，而且可以有无数形式和大小，也没有任何实际的规则。

现在博客已经极大地扩充了互联网的应用和影响，成为了所有网民都可以参与的一种

新媒体，并使得无数的网民有了发言权，有了与政府、机构、企业，以及很多人交流的机会。在博客出现以前，网民是互联网上内容的消费者，网民在互联网上搜寻并下载感兴趣的信息。这些信息是其他人生产的，他们把这些信息放在互联网的某个服务器上，供广大网民使用（也就是供网民消费）。但博客改变了这种情况，网民不仅是互联网上内容的消费者，而且还是互联网上内容的生产者。

从历史上看，weblog 这个新词是 Jorn Barger 于 1997 年创造的。简写的 blog（这是今天最常用的术语）则是 Peter Merholz 于 1999 年创造的。不久，有人把 blog 既当作名词，也当作动词，表示编辑博客或写博客。接着，新名词 blogger 也出现了，它表示博客的拥有者，或博客内容的撰写者和维护者，或博客用户。博客可以看成是继电子邮件、电子公告牌系统 BBS 和即时传信 IM (Instant Messaging)^①之后的第四种网络交流方式。

现在从一些著名的门户网站的主页上都能很容易地进入到博客页面，这让用户查看博客或发表自己的博客都非常方便。前面的图 6-15(b)所示的新浪网站首页，就可看到在几十个分类中的第 1 行第 9 列的“博客”。

当我们在新浪网站主页点击“博客”项时，就可以看到各式各样的博客。也可以利用搜索工具寻找所需的博客。如果我们已在新浪博客注册了，那么也可随时把自己的博客发表在此，让别人来阅读。我们还可直接登录新浪博客网站 blog.sina.com.cn。

博客与个人网站还是有不少区别的。这里最主要的区别就是建立个人网站成本较高，需要租用个人空间、域名等，同时建立网站的个人需要懂得 HTML 语言和网页制作等相关技术；但博客在这方面是不需要什么投资的，所需的技术仅仅是会上网和会用键盘或书写板输入汉字即可。因此网民用较短的时间就能够把自己写的博客发表在网站上，而不像制作个人网站那样花费较多的时间。正因为写博客的门槛较低，广大的网民才有可能成为今天互联网上的信息制造者。

顺便提一下，不要把“博客”和“播客”弄混。播客(Podcast)是苹果手机的一个预装软件，能够让用户通过手机订阅和自动下载所预订的音乐文件，以便随时欣赏音乐。

2. 微博

在图 6-15(b)新浪网站首页各种分类的第 1 行的最后，可以找到“微博”项。微博就是微型博客(microblog)，又称为微博客，它的意思已经非常清楚。博客或微博里的朋友，常称为“博友”。微博也被人戏称为“围脖”，把博友戏称为“脖友”。

但微博不同于一般的博客。微博只记录片段、碎语，三言两语，现场记录，发发感慨，晒晒心情，永远只针对一个问题进行回答。微博只是记录自己琐碎的生活，呈现给人看，而且必须很真实。微博中不必有太多的逻辑思维，很随便，很自由，有点像电影中的一个镜头。写微博比写其他东西简单多了，不需要标题，不需要段落，更不需要漂亮的词汇。

2009 年是中国微博蓬勃发展的一年，相继出现了新浪微博、139 说客、9911、嘀咕网、同学网、贫嘴等微博客。例如，新浪微博就是由中国最大的门户网站新浪网推出的微博服务，是中国目前用户数最多的微博网站(weibo.com)，名人用户众多是新浪微博的一大特色，基本已经覆盖大部分知名文体明星、企业高管、媒体人士。用户可以通过网页、WAP 网、手

^① 注：目前流传的译名还有“即时通信”或“即时通讯”，但 messaging 译为“传信”似乎更准确。

机短信彩信、手机客户端等多种方式更新自己的微博。每条微博字数最初限制为 140 英文字符,但现在已增加了“长微博”的选项,可输入更多的字符。微博还提供插入图片、视频、音乐等功能。截至 2019 年 3 月底,微博的月活跃用户已达 4.65 亿。

现在不少地方政府也开通了微博(即政务微博),这是信息公开的表现。政府可以通过政务微博,及时公布政情、公务、资讯等,获取与民众更多、更直接、更快的沟通,特别是在突发事件或者群体性事件发生的时候,微博就能够成为政府新闻发布的一种重要手段。

虽然政务微博具有“传递信息、沟通上下、解决问题”的功能性特点,并受到广大网民的欢迎,但政务微博的日常管理也非常重要。如果政务微博因缺乏良好的管理而不能够满足群众的各种需求,那么它就会成为一种无用的摆设。

微博是一种互动及传播性极快的工具,其实时性、现场感及快捷性,往往超过所有媒体。这是因为微博对用户的技术要求门槛非常低,而且在语言的编排组织上,没有博客那么高。另外,微博开通的多种 API 使大量的用户可通过手机、网络等方式来即时更新自己的个人信息。微博网站的即时传信功能非常强大,可以通过 QQ 和 MSN 直接书写。

我们正处在一个急剧变革的时代,人们需要用贯穿不同社会阶层的信息去了解社会、改变生活。在互联网上微博的出现正好满足了广大网民的需求。微博发布、转发信息的功能很强大,这种一个人的“通讯社”将对整个社会产生越来越大的影响。

6.4.7 社交网站

社交网站 SNS (Social Networking Site)是近年来发展非常迅速的一种网站,其作用是为一群拥有相同兴趣与活动的人创建在线社区。社交网站的功能非常丰富,如电子邮件、即时传信(在线聊天)、博客撰写、共享相册、上传视频、网页游戏、创建社团、刊登广告等,对现实社交结构已经形成了巨大冲击。社交网络服务提供商针对不同的群众,有着不同的定位,对个人消费者都是免费的。这种网站通过朋友,一传十、十传百地把联系范围不断扩大下去。前面曾提到过的 BBS 和微博,可以看作是社交网站的前身。

2004 年社交网站**脸书**(Facebook,又名面书、脸谱、脸谱网)在美国诞生。脸书最初的用户定位是大学生,但现在它的用户范围已经扩大了很多。接着社交网站热潮席卷全球,而国内以人人网、开心网等为代表的社交网站也如雨后春笋般迅速崛起。社交网站极大地丰富了人们的社交生活,孕育了新的经济增长点,其所蕴含的巨大商业价值和社会力量也正凸显出来。

毫无疑问,目前世界上排名第一且分布最广的社交网站是脸书。脸书最大的特点就是可以非常方便地寻找朋友或联系老同学、老同事,能够简易地在朋友圈中分享图片、视频和音频文件(现在也可以发送其他文件,如.docx, .xlsx 等),以及通过集成的地图功能分享用户所在的位置。现在脸书的月度活跃用户已达 11.5 亿人之多,其中半数以上为移动电话用户。在 2010 年 3 月,脸书在美国的访问人数已超过谷歌,成为全美访问量最大的网站。脸书的官网域名为 Facebook.com,并持有.cn 域名 Facebook.cn。排名第二的社交网站是视频分享网站**油管** YouTube,其月度活跃用户人数为 10 亿人。2006 年 YouTube.com 网站被谷歌收购,目前谷歌手上持有了 youtube.com/.com.cn/.net/.org 等域名。国内类似的视频分享网站有优酷(www.youku.com)、土豆(www.tudou.com)、56 网(56.com)等。

另一种能够提供微博服务的社交网络现在也很流行。例如**推特** Twitter (twitter.com)网站创建于 2006 年,它可以让用户发表不超过 140 个英文字符的消息。这些消息也被称为“推

文”(Tweet)。我国的新浪微博(www.weibo.com)、腾讯微博(t.qq.com)等就是这种性质的社交网站。职业性社交网站领英 LinkedIn 也是很受欢迎的网站。

目前我国最为流行的社交网站就是微信(weixin.qq.com)。微信最初是专为手机用户使用的聊天工具,其功能是“收发信息、拍照分享、联系朋友”。但几年来经过多次系统更新,现在微信不仅可传送文字短信、图片、录音电话、视频短片,还可提供实时音频或视频聊天,甚至可进行网上购物、转账、打车,等等。现在微信的功能已远远超越了社交领域。原来微信仅限于在手机上使用,但新的微信版本已能够安装在普通电脑上。我们知道,电子邮件可以发送给网上任何一个并不认识你的用户,也不管他是否愿意接收你发送的邮件。各种博客和微博也可供任何上网用户浏览。但微信只能在确定的朋友圈中交换信息。正是由于朋友之间更加需要交换信息,而微信的功能又不断在扩展,因此微信在我国已成为几乎每个网民都必备的应用软件。

6.5 电子邮件

6.5.1 电子邮件概述

大家知道,实时通信的电话固然使用方便,但有两个严重缺点。第一,电话通信的主叫和被叫双方必须同时在场。第二,有些电话常常不必要地打断被叫者的工作或休息。

电子邮件(E-mail)是互联网上使用最多的和最受用户欢迎的一种应用。电子邮件把邮件发送到收件人使用的邮件服务器,并放在其中的收件人**邮箱(mail box)**中,收件人可在自己方便时上网到自己使用的邮件服务器进行读取。这相当于互联网为用户设立了存放邮件的信箱,因此 e-mail 有时也称为“**电子信箱**”。电子邮件不仅使用方便,而且还具有传递迅速和费用低廉的优点。据有的公司报道,使用电子邮件后可提高劳动生产率 30%以上。现在电子邮件不仅可传送文字信息,而且还可附上声音和图像。由于电子邮件和手机的广泛使用,现已迫使传统的电报业务退出市场,因为这种传统电报既贵又慢,且很不方便。

1982 年 ARPANET 的电子邮件问世后,很快就成为最受广大网民欢迎的互联网应用。电子邮件的两个最重要的草案标准,是 2008 年更新的**简单邮件传送协议 SMTP (Simple Mail Transfer Protocol)** [RFC 5321]和**互联网文本报文格式**[RFC 5322]。

由于互联网的 SMTP 只能传送可打印的 7 位 ASCII 码邮件,因此在 1996 年又发布了**通用互联网邮件扩充 MIME (Multipurpose Internet Mail Extensions)**[RFC 2045, 草案标准]。MIME 在其邮件首部中说明了邮件的数据类型(如文本、声音、图像、视像等)。在 MIME 邮件中可同时传送多种类型的数据。这在多媒体通信的环境下是非常有用的。

一个电子邮件系统应具有图 6-16 所示的三个主要组成构件,这就是**用户代理**、**邮件服务器**,以及**邮件发送协议**(如 SMTP)和**邮件读取协议**(如 POP3)。POP3 是**邮局协议(Post Office Protocol)**的版本 3。凡是有 TCP 连接的,都经过了互联网,有的甚至可以跨越数千公里的距离。这里为简洁起见,没有画出网络。在互联网中,邮件服务器的数量是很大的。正是这些邮件服务器构成了电子邮件基础结构的核心。在图 6-16 中为了说明问题,仅仅画出了两个邮件服务器。

扫一扫



视频讲解

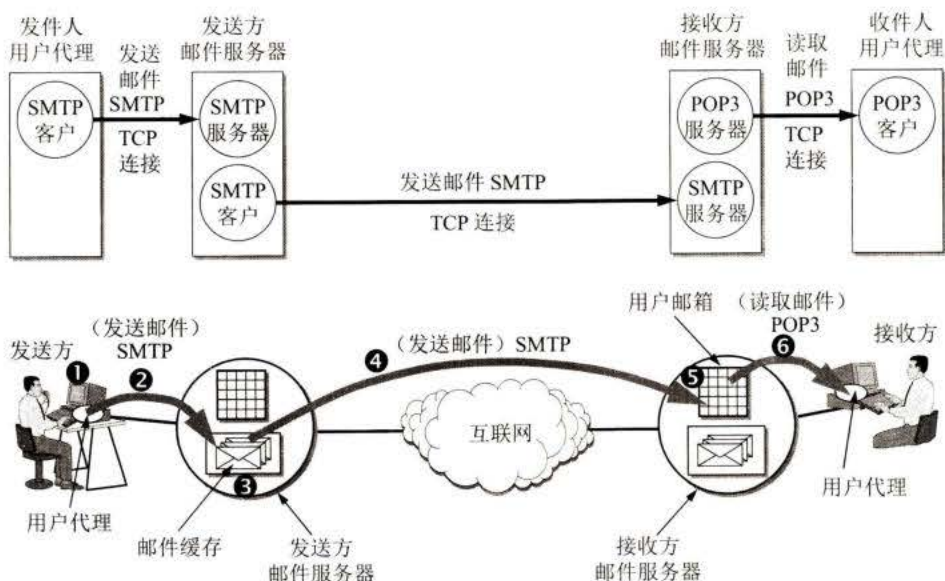


图 6-16 电子邮件的最主要的组成构件

用户代理 UA (User Agent)就是用户与电子邮件系统的接口，在大多数情况下它就是运行在用户计算机中的一个程序。因此用户代理又称为**电子邮件客户端软件**。用户代理向用户提供一个很友好的接口（目前主要是窗口界面）来发送和接收邮件。现在可供大家选择的用户代理有很多种。例如，微软公司的 Outlook Express 和我国张小龙制作的 Foxmail，都是很受欢迎的电子邮件用户代理。

用户代理至少应当具有以下 4 个功能。

(1) **撰写**。给用户提供一个编辑信件的环境。例如，应让用户能创建便于使用的通信录（有常用的人名和地址）。回信时不仅能很方便地从来信中提取出对方地址，并自动地将此地址写入到邮件中合适的位置，而且还能方便地对来信提出的问题答复（系统自动将来信复制一份在用户撰写回信的窗口中，因而用户不需要再输入来信中的问题）。

(2) **显示**。能方便地在计算机屏幕上显示出来信（包括来信附上的声音和图像）。

(3) **处理**。处理包括发送邮件和接收邮件。收件人应根据情况按不同方式对来信进行处理。例如，阅读后删除、存盘、打印、转发等，以及自建目录对来信进行分类保存。有时还可在读取信件之前先查看一下邮件的发件人和长度等，对于不愿收的信件可直接在邮箱中删除。

(4) **通信**。发信人在撰写完邮件后，要利用邮件发送协议将邮件发送到用户所使用的邮件服务器。收件人在接收邮件时，要使用邮件读取协议从本地邮件服务器接收邮件。

互联网上有许多**邮件服务器**可供用户选用（有些要收取少量的邮箱使用费用）。邮件服务器 24 小时不间断地工作，并且具有很大容量的邮件信箱。邮件服务器的功能是发送和接收邮件，同时还要向发件人报告邮件传送的结果（已交付、被拒绝、丢失等）。邮件服务器按照客户服务器方式工作。邮件服务器需要使用**两种不同的协议**。一种协议用于用户代理向邮件服务器发送邮件或在邮件服务器之间发送邮件，如 SMTP 协议，而另一种协议用于用户代理从邮件服务器读取邮件，如**邮局协议 POP3**。

这里应当注意，邮件服务器必须能够同时充当客户和服务器。例如，当邮件服务器 A

向另一个邮件服务器 B 发送邮件时, A 就作为 SMTP 客户, 而 B 是 SMTP 服务器。反之, 当 B 向 A 发送邮件时, B 就是 SMTP 客户, 而 A 就是 SMTP 服务器。

图 6-16 给出了计算机之间发送和接收电子邮件的几个重要步骤。请注意, SMTP 和 POP3 (或 IMAP) 都是使用 TCP 连接来传送邮件的, 使用 TCP 的目的是为了可靠地传送邮件。

❶ 发件人调用计算机中的用户代理撰写和编辑要发送的邮件。

❷ 发件人点击屏幕上的“发送邮件”按钮, 把发送邮件的工作全都交给用户代理来完成。用户代理把邮件用 SMTP 协议发给发送方邮件服务器, 用户代理充当 SMTP 客户, 而发送方邮件服务器充当 SMTP 服务器。用户代理所进行的这些工作, 用户是看不到的。有的用户代理可以让用户在屏幕上看见邮件发送的进度显示。用户所使用的邮件服务器究竟在什么地方, 用户并不知道, 也没必要知道。实际上, 用户在把写好的信件交付给用户代理后, 就什么都不用管了。

❸ SMTP 服务器收到用户代理发来的邮件后, 就把邮件临时存放在邮件缓存队列中, 等待发送到接收方的邮件服务器 (等待时间的长短取决于邮件服务器的处理能力和队列中待发送的信件的数量。但这种等待时间一般都远远大于分组在路由器中等待转发的排队时间)。

❹ 发送方邮件服务器的 SMTP 客户与接收方邮件服务器的 SMTP 服务器建立 TCP 连接, 然后就把邮件缓存队列中的邮件依次发送出去。请注意, 邮件不会在互联网中的某个中间邮件服务器落地。如果 SMTP 客户还有一些邮件要发送到同一个邮件服务器, 那么可以在原来已建立的 TCP 连接上重复发送。如果 SMTP 客户无法和 SMTP 服务器建立 TCP 连接 (例如, 接收方服务器过负荷或出了故障), 那么要发送的邮件就会继续保存在发送方的邮件服务器中, 并在稍后一段时间再进行新的尝试。如果 SMTP 客户超过了规定的时间还不能把邮件发送出去, 那么发送邮件服务器就把这种情况通知用户代理。

❺ 运行在接收方邮件服务器中的 SMTP 服务器进程收到邮件后, 把邮件放入收件人的用户邮箱中, 等待收件人进行读取。

❻ 收件人在打算收信时, 就运行计算机中的用户代理, 使用 POP3 (或 IMAP) 协议读取发送给自己的邮件。请注意, 在图 6-16 中, POP3 服务器和 POP3 客户之间的箭头表示的是邮件传送的方向。但它们之间的通信是由 POP3 客户发起的。

请注意这里有两种不同的通信方式。一种是“推”(push): SMTP 客户把邮件“推”给 SMTP 服务器。另一种是“拉”(pull): POP3 客户把邮件从 POP3 服务器“拉”过来。细心的读者可能会想到这样的问题: 如果让图 6-16 中的邮件服务器程序就在发送方和接收方的计算机中运行, 那么岂不是可以直接把邮件发送到收件人的计算机中?

答案是“不行”。这是因为并非所有的计算机都能运行邮件服务器程序。有些计算机可能没有足够的存储空间来运行允许程序在后台运行的操作系统, 或是可能没有足够的 CPU 能力来运行邮件服务器程序。更重要的是, 邮件服务器程序必须不间断地运行, 每天 24 小时都必须不间断地连接在互联网上, 否则就可能使很多外面发来的邮件无法接收。这样看来, 让用户的计算机运行邮件服务器程序显然是很不现实的 (一般用户在不使用计算机时就将机器关闭)。让来信暂时存储在用户的邮件服务器中, 而当用户方便时就从邮件服务器的用户信箱中读取来信, 则是一种比较合理的做法。在 Foxmail 中使用一种“特快专递”服务。这种服务就是从发件人的用户代理直接利用 SMTP 把邮件发送到接收方邮件服务器。这就加快了邮件的交付 (省去在发送方邮件服务器中的排队等待时间)。但这种“特快专递”和邮

政的 EMS 直接把邮件送到用户家中不同，它并没有把邮件直接发送到收件人的计算机中。但有些邮件服务器为了防止垃圾邮件和计算机病毒，拒绝接收从一般用户直接发来的邮件。

电子邮件由信封(envelope)和内容(content)两部分组成。电子邮件的传输程序根据邮件信封上的信息来传送邮件。这与邮局按照信封上的信息投递信件是相似的。

在邮件的信封上，最重要的就是收件人的地址。TCP/IP 体系的电子邮件系统规定**电子邮件地址**(E-mail address)的格式如下：

用 户 名 @ 邮 件 服 务 器 的 域 名 (6-1)

在式(6-1)中，符号“@”应读作“at”，表示“在”的意思。例如，在电子邮件地址“xyz@abc.com”中，“abc.com”就是邮件服务器的域名，而“xyz”就是在这个邮件服务器中收件人的用户名，也就是**收件人邮箱名**，是收件人为自己定义的字符串标识符。但应注意，这个用户名在邮件服务器中必须是唯一的（当用户定义自己的用户名时，邮件服务器要负责检查该用户名在本服务器中的唯一性）。这样就保证了每一个电子邮件地址在世界范围内是唯一的。这对保证电子邮件能够在整个互联网范围内的准确交付是十分重要的。电子邮件的用户一般采用容易记忆的字符串。

6.5.2 简单邮件传送协议 SMTP

下面介绍 SMTP 的一些主要特点。

SMTP 规定了在两个相互通信的 SMTP 进程之间应如何交换信息。由于 SMTP 使用客户服务器方式，因此负责发送邮件的 SMTP 进程就是 SMTP 客户，而负责接收邮件的 SMTP 进程就是 SMTP 服务器。至于邮件内部的格式，邮件如何存储，以及邮件系统应以多快的速度来发送邮件，SMTP 也都未做出规定。

SMTP 规定了 14 条命令和 21 种应答信息。每条命令用几个字母组成，而每一种应答信息一般只有一行信息，由一个 3 位数字的代码开始，后面附上（也可不附上）很简单的文字说明。下面通过发送方和接收方的邮件服务器之间的 SMTP 通信的三个阶段介绍几个最主要的命令和响应信息。

1. 连接建立

发件人的邮件送到发送方邮件服务器的邮件缓存后，SMTP 客户就每隔一定时间（例如 30 分钟）对邮件缓存扫描一次。如发现有邮件，就使用 SMTP 的熟知端口号码 25 与接收方邮件服务器的 SMTP 服务器建立 TCP 连接。在连接建立后，接收方 SMTP 服务器要发出“220 Service ready”（服务就绪）。然后 SMTP 客户向 SMTP 服务器发送 HELO 命令，附上发送方的主机名。SMTP 服务器若有能力接收邮件，则回答：“250 OK”，表示已准备好接收。若 SMTP 服务器不可用，则回答“421 Service not available”（服务不可用）。

如在一定时间内（例如三天）发送不了邮件，邮件服务器会把这个情况通知发件人。

SMTP 不使用中间的邮件服务器。不管发送方和接收方的邮件服务器相隔有多远，不管在邮件传送过程中要经过多少个路由器，TCP 连接总是在发送方和接收方这两个邮件服务器之间直接建立。当接收方邮件服务器出故障而不能工作时，发送方邮件服务器只能等待一段时间后再尝试和该邮件服务器建立 TCP 连接，而不能先找一个中间的邮件服务器建立 TCP 连接。

2. 邮件传送

邮件的传送从 MAIL 命令开始。MAIL 命令后面有发件人的地址。如：MAIL FROM: <xiexiren@tsinghua.org.cn>。若 SMTP 服务器已准备好接收邮件，则回答“250 OK”。否则，返回一个代码，指出原因。如：451（处理时出错）、452（存储空间不够）、500（命令无法识别）等。

下面跟着一个或多个 RCPT 命令，取决于把同一个邮件发送给一个或多个收件人，其格式为 RCPT TO: <收件人地址>。RCPT 是 recipient（收件人）的缩写。每发送一个 RCPT 命令，都应当有相应的信息从 SMTP 服务器返回，如：“250 OK”，表示指明的邮箱在接收方的系统中，或“550 No such user here”（无此用户），即不存在此邮箱。

RCPT 命令的作用就是：先弄清接收方系统是否已做好接收邮件的准备，然后才发送邮件。这样做是为了避免浪费通信资源，不至于发送了很长的邮件以后才知道地址错误。

再下面就是 DATA 命令，表示要开始传送邮件的内容了。SMTP 服务器返回的信息是：“354 Start mail input; end with <CRLF>.<CRLF>”。这里<CRLF>是“回车换行”的意思。若不能接收邮件，则返回 421（服务器不可用），500（命令无法识别）等。接着 SMTP 客户就发送邮件的内容。发送完毕后，再发送<CRLF>.<CRLF>（两个回车换行中间用一个点隔开）表示邮件内容结束。实际上在服务器端看到的可打印字符只是一个英文的句点。若邮件收到了，则 SMTP 服务器返回信息“250 OK”，或返回差错代码。

虽然 SMTP 使用 TCP 连接试图使邮件的传送可靠，但“发送成功”并不等于“收件人读取了这个邮件”。当一个邮件传送到接收方的邮件服务器后（即接收方的邮件服务器收下了这个邮件），再往后的情况如何，就有以下几种可能性：

(1) 接收方的邮件服务器刚收到邮件后就出了故障，使收到的邮件全部丢失（在收件人读取信件之前）。

(2) 收件人由于某种原因，未检查自己的邮箱，不知道邮箱中有新的来信。

(3) 收件人很长时间不删除邮箱中的过期邮件，导致有限的邮箱容量用尽，无法再接收新的邮件。

(4) 邮件服务器根据某种原则，误判某些邮件属于垃圾邮件，就把收到的某些邮件转入到垃圾邮件的文件夹。收件人应从垃圾邮件的文件夹中清理出未收到的非垃圾邮件。

因此，使用电子邮件的人，应当养成及时收取和清理自己的邮箱的好习惯。

3. 连接释放

邮件发送完毕后，SMTP 客户应发送 QUIT 命令。SMTP 服务器返回的信息是“221（服务关闭）”，表示 SMTP 同意释放 TCP 连接。邮件传送的全部过程即结束。

这里再强调一下，使用电子邮件的用户看不见以上这些过程，所有这些复杂过程都被电子邮件的用户代理屏蔽了。

已经广泛使用多年的 SMTP 存在着一些缺点。例如，发送电子邮件不需要经过鉴别。这就是说，在 FROM 命令后面的地址可以任意填写。这就大大方便了垃圾邮件的作者，给收信人添加了麻烦（有人估计，在全世界所有的电子邮件中，垃圾邮件至少占到 50%以上，甚至高达 90%）。又如，SMTP 本来就是为传送 ASCII 码而不是传送二进制数据设计的。虽然后来有了 MIME 可以传送二进制数据（见后面 6.5.6 节的介绍），但在传送非 ASCII 码的长报文时，在网络上的传输效率是不高的。此外，SMTP 传送的邮件是明文，不利于保密。

为了解决上述问题，2008 年 10 月颁布的草案标准 RFC 5321 对 SMTP 进行了扩充，成为扩充的 SMTP (Extended SMTP)，记为 ESMTP。RFC 5321 在许多命令中增加了扩展的参数。新增加的功能有：客户端的鉴别，服务器接受二进制报文，服务器接受分块传送的大报文，发送前先检查报文的大小，使用安全传输 TLS（见下一章 7.6.2 节），以及使用国际化地址等。考虑到现在的许多 SMTP 邮件服务器可能还没有升级到 ESMTP，因此特规定使用 ESMTP 的客户端在准备传送报文时，不是发送 HELO 而是发送 EHLO 报文。如果 EHLO 报文被对方服务器端拒绝，就表明对方仍然是一个标准的 SMTP 邮件服务器（不使用扩展的参数），因而就要按照原来使用的 SMTP 参数进行邮件的传送。如果 EHLO 报文被接受了，那么客户端就可以使用 ESMTP 扩展的参数传送报文了。

6.5.3 电子邮件的信息格式

一个电子邮件分为信封和内容两大部分。在草案标准 RFC 5322 文档中只规定了邮件内容中的首部(header)格式，而对邮件的主体(body)部分则让用户自由撰写。用户写好首部后，邮件系统自动地将信封所需的信息提取出来并写在信封上。所以用户不需要填写电子邮件信封上的信息。

邮件内容首部包括一些关键字，后面加上冒号。最重要的关键字是：To 和 Subject。

“To:”后面填入一个或多个收件人的电子邮件地址。在电子邮件软件中，用户把经常通信的对象姓名和电子邮件地址写到地址簿(address book)中。当撰写邮件时，只需打开地址簿，点击收件人名字，收件人的电子邮件地址就会自动地填入到合适的位置上。

“Subject:”是邮件的主题。它反映了邮件的主要内容。主题类似于文件系统的文件名，便于用户查找邮件。

邮件首部还有一项是抄送“Cc:”。这两个字符来自“Carbon copy”，意思是留下一个“复写副本”。这是借用旧的名词，表示应给某某人发送一个邮件副本。

有些邮件系统允许用户使用关键字 Bcc (Blind carbon copy)来实现盲复写副本。这是使发件人能将邮件的副本送给某人，但不希望此事为收件人知道。Bcc 又称为暗送。

首部关键字还有“From”和“Date”，表示发件人的电子邮件地址和发信日期。这两项一般都由邮件系统自动填入。

另一个关键字是“Reply-To”，即对方回信所用的地址。这个地址可以与发件人发信时所用的地址不同。例如有时到外地借用他人的邮箱给自己的朋友发送邮件，但仍希望对方将回信发送到自己的邮箱。这一项可以事先设置好，不需要在每次写信时进行设置。

6.5.4 邮件读取协议 POP3 和 IMAP

现在常用的邮件读取协议有两个，即邮局协议第 3 个版本 POP3 和网际报文存取协议 IMAP (Internet Message Access Protocol)。现分别讨论如下。

邮局协议 POP 是一个非常简单、但功能有限的邮件读取协议。邮局协议 POP 最初公布于 1984 年。经过几次更新，现在使用的是 1996 年的版本 POP3 [RFC 1939, STD53]，大多数的 ISP 都支持 POP3。

POP3 也使用客户服务器的工作方式。在接收邮件的用户计算机中的用户代理必须运行 POP3 客户程序，而在收件人所连接的 ISP 的邮件服务器中则运行 POP3 服务器程序。当然，这个 ISP 的邮件服务器还必须运行 SMTP 服务器程序，以便接收发送方邮件服务器的 SMTP

客户程序发来的邮件。这些请参阅图 6-16。POP3 服务器只有在用户输入鉴别信息（用户名和口令）后，才允许对邮箱进行读取。

POP3 协议的一个特点就是只要用户从 POP3 服务器读取了邮件，POP3 服务器就把该邮件删除。这在某些情况下就不够方便。例如，某用户在办公室的台式计算机上接收了一个邮件，还来不及写回信，就马上携带笔记本电脑出差。当他打开笔记本电脑写回信时，POP3 服务器上却已经删除了原来已经看过的邮件（除非他事先将这些邮件复制到笔记本电脑中）。为了解决这一问题，POP3 进行了一些功能扩充，其中包括让用户能够事先设置邮件读取后仍然在 POP3 服务器中存放的时间[RFC 2449，建议标准]。

另一个读取邮件的协议是网际报文存取协议 IMAP，它比 POP3 复杂得多。IMAP 和 POP 都按客户服务器方式工作，但它们有很大的差别。现在较新的版本是 2003 年 3 月修订的版本 4，即 IMAP4 [RFC 3501，建议标准]。不过在习惯上，对这个协议大家很少加上版本号“4”，而经常简单地用 IMAP 表示 IMAP4。但是对 POP3 却不会忘记写上版本号“3”。

在使用 IMAP 时，在用户的计算机上运行 IMAP 客户程序，然后与接收方的邮件服务器上的 IMAP 服务器程序建立 TCP 连接。用户在自己的计算机上就可以操纵邮件服务器的邮箱，就像在本地操纵一样，因此 IMAP 是一个联机协议。当用户计算机上的 IMAP 客户程序打开 IMAP 服务器的邮箱时，用户就可看到邮件的首部。若用户需要打开某个邮件，则该邮件才传到用户的计算机上。用户可以根据需要为自己的邮箱创建便于分类管理的层次式的邮箱文件夹，并且能够将存放的邮件从某一个文件夹中移动到另一个文件夹中。用户也可按某种条件对邮件进行查找。在用户未发出删除邮件的命令之前，IMAP 服务器邮箱中的邮件一直保存着。

IMAP 最大的好处就是用户可以在不同的地方使用不同的计算机（例如，使用办公室的计算机、或家中的计算机，或在外地使用笔记本电脑）随时上网阅读和处理自己在邮件服务器中的邮件。IMAP 还允许收件人只读取邮件中的某一个部分。例如，收到了一个带有视像附件（此文件可能很大）的邮件，而用户使用的是无线上网，信道的传输速率很低。为了节省时间，可以先下载邮件的正文部分，待以后有时间再读取或下载这个很大的附件。

IMAP 的缺点是如果用户没有将邮件复制到自己的计算机上，则邮件一直存放在 IMAP 服务器上。要想查阅自己的邮件，必须先上网。

下面的表 6-2 给出了 IMAP 和 POP3 的主要功能的比较。

表 6-2 IMAP 和 POP3 的主要功能比较

操作位置	操作内容	IMAP	POP3
收件箱	阅读、标记、移动、删除邮件等	客户端与邮箱更新同步	仅在客户端内
发件箱	保存到已发送	客户端与邮箱更新同步	仅在客户端内
创建文件夹	新建自定义的文件夹	客户端与邮箱更新同步	仅在客户端内
草稿	保存草稿	客户端与邮箱更新同步	仅在客户端内
垃圾文件夹	接收并移入垃圾文件夹的邮件	支持	不支持
广告邮件	接收并移入广告邮件夹的邮件	支持	不支持

最后再强调一下，不要把邮件读取协议 POP3 或 IMAP 与邮件传送协议 SMTP 弄混。发件人的用户代理向发送方邮件服务器发送邮件，以及发送方邮件服务器向接收方邮件服务器发送邮件，都是使用 SMTP 协议。而 POP3 或 IMAP 则是用户代理从接收方邮件服务器上读

取邮件所使用的协议。

6.5.5 基于万维网的电子邮件

从前面的图 6-16 可看出,用户要使用电子邮件,必须在自己使用的计算机中安装用户代理软件 UA。如果外出到某地而又未携带自己的笔记本电脑,那么要使用别人的计算机进行电子邮件的收发,将是非常不方便的。

现在这个问题解决了。在 20 世纪 90 年代中期,Hotmail 推出了基于万维网的电子邮件(Webmail)。今天,几乎所有的著名网站以及大学或公司,都提供了万维网电子邮件。常用的万维网电子邮件有谷歌的 Gmail (@gmail.com),微软的 Hotmail (@hotmail.com),雅虎的 Yahoo 邮箱 (@yahoo.com)。我国的互联网技术公司也都提供万维网邮件服务,例如,网易邮箱 (@163.com 或 @126.com)、新浪邮箱 (@sina.com 或 @sina.cn)和腾讯的 QQ 邮箱 (@qq.com)等。

万维网电子邮件的好处就是:不管在什么地方(在任何一个国家的网吧、宾馆或朋友家中),只要能够找到上网的计算机,在打开任何一种浏览器后,就可以非常方便地收发电子邮件。使用万维网电子邮件不需要在计算机中再安装用户代理软件。浏览器本身可以向用户提供非常友好的电子邮件界面(和原来的用户代理提供的界面相似),使用户在浏览器上就能够很方便地撰写和收发电子邮件。

例如,你使用的是网易的 163 邮箱,那么在任何一个浏览器的地址栏中,键入 163 邮箱的 URL (mail.163.com),按回车键后,就可以使用 163 电子邮件了,这在家中一样方便。你曾经接收和发送过的邮件、已删除的邮件以及你的通信录等内容,都照常呈现在屏幕上。

我们知道,用户在浏览器中浏览各种信息时需要使用 HTTP 协议。因此,在浏览器和互联网上的邮件服务器之间传送邮件时,仍然使用 HTTP 协议。但是在各邮件服务器之间传送邮件时,则仍然使用 SMTP 协议。

6.5.6 通用互联网邮件扩充 MIME

1. MIME 概述

前面所述的电子邮件协议 SMTP 有以下缺点:

(1) SMTP 不能传送可执行文件或其他的二进制对象。人们曾试图将二进制文件转换为 SMTP 使用的 ASCII 文本,例如流行的 UNIX UUencode/UUdecode 方案,但这些均未形成正式标准或事实上的标准。

(2) SMTP 限于传送 7 位的 ASCII 码。许多其他非英语国家的文字(如中文、俄文,甚至带重音符号的法文或德文)就无法传送。即使在 SMTP 网关将 EBCDIC 码(即扩充的二/十进制交换码)转换为 ASCII 码,也会遇到一些麻烦。

(3) SMTP 服务器会拒绝超过一定长度的邮件。

(4) 某些 SMTP 的实现并没有完全按照 SMTP 的互联网标准。常见的问题如下:

- 回车、换行的删除和增加;
- 超过 76 个字符时的处理:截断或自动换行;
- 后面多余空格的删除;

- 将制表符 tab 转换为若干个空格。

于是在这种情况下就提出了通用互联网邮件扩充 MIME [RFC 2045~2049, 前三个文档是草案标准]。MIME 并没有改动或取代 SMTP。MIME 的意图是继续使用原来的邮件格式, 但增加了邮件主体的结构, 并定义了传送非 ASCII 码的编码规则。也就是说, MIME 邮件可在现有的电子邮件程序和协议下传送。图 6-17 表示 MIME 和 SMTP 的关系。



图 6-17 MIME 和 SMTP 的关系

MIME 主要包括以下三部分内容：

(1) 5 个新的邮件首部字段，它们可包含在原来的邮件首部中。这些字段提供了有关邮件主体的信息。

(2) 定义了许多邮件内容的格式，对多媒体电子邮件的表示方法进行了标准化。

(3) 定义了传送编码，可对任何内容格式进行转换，而不会被邮件系统改变。

为适应于任意数据类型和表示，每个 MIME 报文包含告知收件人数据类型和使用编码的信息。MIME 把增加的信息加入到原来的邮件首部中。

下面是 MIME 增加的 5 个新的邮件首部的名称及其意义（有的可以是选项）。

(1) MIME-Version: 标志 MIME 的版本。现在的版本号是 1.0。若无此行，则为英文文本。

(2) Content-Description: 这是可读字符串，说明此邮件主体是否是图像、音频或视频。

(3) Content-Id: 邮件的唯一标识符。

(4) Content-Transfer-Encoding: 在传送时邮件的主体是如何编码的。

(5) Content-Type: 说明邮件主体的数据类型和子类型。

上述的前三项的意思很清楚，因此下面只对后两项进行介绍。

2. 内容传送编码

下面介绍三种常用的内容传送编码 (Content-Transfer-Encoding)。

最简单的编码就是 7 位 ASCII 码，而每行不能超过 1000 个字符。MIME 对这种由 ASCII 码构成的邮件主体不进行任何转换。

另一种编码称为 quoted-printable，这种编码方法适用于所传送的数据中只有少量的非 ASCII 码，例如汉字。这种编码方法的要点就是对于所有可打印的 ASCII 码，除特殊字符等号 “=” 外，都不改变。等号 “=” 和不可打印的 ASCII 码以及非 ASCII 码的数据的编码方法是：先将每个字节的二进制代码用两个十六进制数字表示，然后在前面再加上一个等号 “=”。例如，汉字的“系统”的二进制编码是：11001111 10110101 11001101 10110011（共有 32 位，但这四个字节都不是 ASCII 码），其十六进制数字表示为：CFB5CD B3。用 quoted-printable 编码表示为：=CF=B5=CD=B3，这 12 个字符都是可打印的 ASCII 字符，它

们的二进制编码^①需要 96 位，和原来的 32 位相比，开销达 200%。而等号“=”的二进制代码为 00111101，即十六进制的 3D，因此等号“=”的 quoted-printable 编码为“=3D”。

对于任意的二进制文件，可用 base64 编码。这种编码方法是先把二进制代码划分为一个个 24 位长的单元，然后把每一个 24 位单元划分为 4 个 6 位组。每一个 6 位组按以下方法转换成 ASCII 码。6 位的二进制代码共有 64 种不同的值，从 0 到 63。用 A 表示 0，用 B 表示 1，等等。26 个大写字母排列完毕后，接下去再排 26 个小写字母，再后面是 10 个数字，最后用“+”表示 62，而用“/”表示 63。再用两个连在一起的等号“==”和一个等号“=”分别表示最后一组的代码只有 8 位或 16 位。回车和换行都忽略，它们可在任何地方插入。

下面是一个 base64 编码的例子：

24 位二进制代码	01001001 00110001 01111001
划分为 4 个 6 位组	010010 010011 000101 111001
对应的 base64 编码	S T F 5
用 ASCII 编码发送	01010011 01010100 01000110 00110101

不难看出，24 位的二进制代码采用 base64 编码后变成了 32 位，开销为 8 位，占 32 位的 25%。

3. 内容类型

MIME 标准规定 Content-Type 说明必须含有两个标识符，即内容类型(type)和子类型(subtype)，中间用“/”分开。

MIME 一直到现在还是草案标准。除了内容类型和子类型，MIME 允许发件人和收件人自己定义专用的内容类型。但为避免可能出现名字冲突，标准要求为专用的内容类型选择的名称要以字符串 X-开始。后来陆续出现了几百个子类型，而且子类型的数目还在不断地增加。现在可以在网站上查出现有的 MIME 类型和子类型的名称，以及申请新的子类型的具体步骤[W-MEDIA-TYPE]。表 6-3 列出了 MIME 常用的内容类型、子类型举例及其说明^②。

表 6-3 可出现在 MIME Content-Type 说明中的类型及子类型举例

内容类型	子类型举例	说明
text (文本)	plain, html, xml, css	不同格式的文本
image (图像)	gif, jpeg, tiff	不同格式的静止图像
audio (音频)	basic, mpeg, mp4	可听见的声音
video (视频)	mpeg, mp4, quicktime	不同格式的影片

① 注：ASCII 码原来定义为 7 位码。但最初为了增加检错能力就增加了一个奇偶检验位，因而使用一个字节（8 位）表示一个 ASCII 码。于是 ASCII 码就变成了 8 位码，但其最高位一定是 0，而后面的 7 位就是最初定义的 ASCII 编码。

② 注：GIF (Graphics Interchange Format), JPEG (Joint Photographic Expert Group) 和 TIFF (Tagged Image File Format)都是静止图像（如照片）格式的标准，前两者是压缩的，后者可以是压缩的，也可以是不压缩的。而 MPEG (Motion Picture Experts Group)是活动图像（如电影）的压缩标准，包括 MPEG 视频、MPEG 音频和 MPEG 系统（视频音频同步）三个部分。QuickTime 是苹果公司的媒体播放机。VRML (Virtual Reality Modeling Language)是虚拟现实建模语言。PDF (Portable Document Format)是 Adobe 公司开发的一种电子文件格式。ZIP 是一种文件压缩算法。

内容类型	子类型举例	说明
model (模型)	vrmf	3D 模型
application (应用)	octet-stream, pdf, javascript, zip	不同应用程序产生的数据
message (报文)	http, rfc822	封装的报文
multipart (多部分)	mixed, alternative, parallel, digest	多种类型的组合

MIME 的内容类型中的 **multipart** 是很有用的，因为它使邮件增加了相当大的灵活性。MIME 标准为 **multipart** 定义了四种可能的子类型，每个子类型都提供重要功能。

(1) **mixed** 子类型允许单个报文含有多个相互独立的子报文，每个子报文可有自己的类型和编码。**mixed** 子类型报文使用户能够在单个报文中附上文本、图形和声音，或者用额外数据段发送一个备忘录，类似商业信笺含有的附件。在 **mixed** 后面还要用到一个关键字，即 **Boundary=**，此关键字定义了分隔报文各部分所用的字符串（由邮件系统定义），只要在邮件的内容中不会出现这样的字符串即可。当某一行以两个连字符“--”开始，后面紧跟上述的字符串，就表示下面开始了另一个子报文。

(2) **alternative** 子类型允许单个报文含有同一数据的多种表示。当给多个使用不同硬件和软件系统的收件人发送备忘录时，这种类型的 **multipart** 报文就很有用。例如，用户可同时用普通的 ASCII 文本和格式化的形式发送文本，从而允许拥有图形功能的计算机用户在查看图形时选择格式化的形式。

(3) **parallel** 子类型允许单个报文含有可同时显示的各个子部分（例如，图像和声音子部分必须一起播放）。

(4) **digest** 子类型允许单个报文含有一组其他报文（如从讨论中收集电子邮件报文）。

下面显示了一个 MIME 邮件，它包含有一个简单解释的文本和含有非文本信息的照片。邮件中第一部分的注解说明第二部分含有一张照片。

```
From: xiexiren@tsinghua.org.cn
To: xyz@163.com
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary=qwertyuiop
```

```
--qwertyuiop
XYZ:
```

你要的图片在此邮件中，收到后请回信。

谢希仁

```
--qwertyuiop
Content-Type: image/gif
Content-Transfer-Encoding: base64
...data for the image (图像的数据)...
--qwertyuiop--
```

上面最后一行表示 **boundary** 的字符串后面还有两个连字符“--”，表示整个 **multipart** 的结束。



6.6 动态主机配置协议 DHCP

为了把协议软件做成通用的和便于移植的, 协议软件的编写者不会把所有的细节都固定在源代码中。相反, 他们把协议软件参数化。这就使得在很多台计算机上有可能使用同一个经过编译的二进制代码。一台计算机和另一台计算机的许多区别, 都可以通过一些不同的参数来体现。在协议软件运行之前, 必须给每一个参数赋值。

在协议软件中给这些参数赋值的动作叫作**协议配置**。一个协议软件在使用之前必须是已正确配置的。具体的配置信息有哪些则取决于协议栈。例如, 连接到互联网的计算机的协议软件需要配置的项目包括:

- (1) IP 地址;
- (2) 子网掩码;
- (3) 默认路由器的 IP 地址;
- (4) 域名服务器的 IP 地址。

为了省去给计算机配置 IP 地址的麻烦, 我们能否在计算机的生产过程中, 事先给每一台计算机配置好一个唯一的 IP 地址呢(如同每一个以太网适配器拥有一个唯一的 MAC 地址)? 这显然是不行的。这是因为 IP 地址不仅包括了主机号, 而且还包括了网络号。一个 IP 地址指出了一台计算机连接在哪一个网络上。当计算机还在生产时, 无法知道它在出厂后将被连接到哪一个网络上。因此, 需要连接到互联网的计算机, 必须对 IP 地址等项目进行协议配置。

用人工进行协议配置很不方便, 而且容易出错。因此, 应当采用自动协议配置的方法。

互联网现在广泛使用的是**动态主机配置协议 DHCP** (Dynamic Host Configuration Protocol), 它提供了一种机制, 称为**即插即用连网**(plug-and-play networking)。这种机制允许一台计算机加入新的网络和获取 IP 地址而不用手工参与。DHCP 最新的 RFC 文档还是互联网草案标准[RFC 2131—2132]。

DHCP 对运行客户软件和服务器软件的计算机都适用。当运行客户软件的计算机移到一个新的网络时, 就可使用 DHCP 获取其配置信息而不需要手工干预。DHCP 给运行服务器软件而位置固定的计算机指派一个永久地址, 而当这计算机重新启动时其地址不改变。

DHCP 使用客户服务器方式。需要 IP 地址的主机在启动时就向 DHCP 服务器广播发送**发现报文**(DHCPDISCOVER) (将目的 IP 地址置为全 1, 即 255.255.255.255), 这时该主机就成为 DHCP 客户。发送广播报文是因为现在还不知道 DHCP 服务器在什么地方, 因此要发现(DHCPDISCOVER)DHCP 服务器的 IP 地址。这台主机目前还没有自己的 IP 地址, 因此它将 IP 数据报的源 IP 地址设为全 0。这样, 在本地网络上的所有主机都能够收到这个广播报文, 但只有 DHCP 服务器才对此广播报文进行回答。DHCP 服务器先在其数据库中查找该计算机的配置信息。若找到, 则返回找到的信息。若找不到, 则从服务器的 IP 地址池(address pool)中取一个地址分配给该计算机。DHCP 服务器的回答报文叫作**提供报文**(DHCPOFFER), 表示“提供”了 IP 地址等配置信息。

但是我们并不愿意在每一个网络上都设置一个 DHCP 服务器, 因为这样会使 DHCP 服务器的数量太多。因此现在是使每一个网络至少有一个 DHCP **中继代理**(relay agent) (通常是一台路由器, 如图 6-18 所示), 它配置了 DHCP 服务器的 IP 地址信息。当 DHCP 中继代理收到主机 A 以**广播**形式发送的发现报文后, 就以**单播**方式向 DHCP 服务器转发此报文,

并等待其回答。收到 DHCP 服务器回答的提供报文后，DHCP 中继代理再把此提供报文发回给主机 A。需要注意的是，图 6-18 只是个示意图。实际上，DHCP 报文只是 UDP 用户数据报的数据，它还要加上 UDP 首部、IP 数据报首部，以及以太网的 MAC 帧的首部和尾部后，才能在链路上传送。



图 6-18 DHCP 中继代理以单播方式转发发现报文

DHCP 服务器分配给 DHCP 客户的 IP 地址是临时的，因此 DHCP 客户只能在一段有限的时间内使用这个分配到的 IP 地址。DHCP 协议称这段时间为租用期(lease period)，但并没有具体规定租用期应取为多长或至少为多长，这个数值应由 DHCP 服务器自己决定。例如，一个校园网的 DHCP 服务器可将租用期设定为 1 小时。DHCP 服务器在给 DHCP 发送的提供报文的选项中给出租用期的数值。按照 RFC 2132 的规定，租用期用 4 字节的二进制数字表示，单位是秒。因此可供选择的租用期范围从 1 秒到 136 年。DHCP 客户也可在自己发送的报文中（例如，发现报文）提出对租用期的要求。

DHCP 的详细工作过程如图 6-19 所示。DHCP 客户使用的 UDP 端口是 68，而 DHCP 服务器使用的 UDP 端口是 67。这两个 UDP 端口都是熟知端口。



图 6-19 DHCP 协议的工作过程

下面按照图 6-19 中的注释编号（①至⑨）进行简单的解释。

- ① DHCP 服务器被动打开 UDP 端口 67，等待客户端发来的报文。
- ② DHCP 客户从 UDP 端口 68 发送 DHCP 发现报文。
- ③ 凡收到 DHCP 发现报文的 DHCP 服务器都发出 DHCP 提供报文，因此 DHCP 客户



视频讲解

可能收到多个 DHCP 提供报文。

- ④ DHCP 客户从几个 DHCP 服务器中选择其中的一个，并向所选择的 DHCP 服务器发送 DHCP 请求报文。

- ⑤ 被选择的 DHCP 服务器发送确认报文 DHCPACK。从这时起，DHCP 客户就可以使用这个 IP 地址了。这种状态叫作**已绑定状态**，因为在 DHCP 客户端的 IP 地址和 MAC 地址已经完成绑定，并且可以开始使用得到的临时 IP 地址了。

DHCP 客户现在要根据服务器提供的租用期 T 设置两个计时器 T_1 和 T_2 ，它们的超时时间分别是 $0.5T$ 和 $0.875T$ 。当超时时间到了就要请求更新租用期。

- ⑥ 租用期过了一半（ T_1 时间到），DHCP 客户发送请求报文 DHCPREQUEST 要求更新租用期。

- ⑦ DHCP 服务器若同意，则发回确认报文 DHCPACK。DHCP 客户得到了新的租用期，重新设置计时器。

- ⑧ DHCP 服务器若不同意，则发回否认报文 DHCPNACK。这时 DHCP 客户必须立即停止使用原来的 IP 地址，而必须重新申请 IP 地址（回到步骤②）。

若 DHCP 服务器不响应步骤⑥的请求报文 DHCPREQUEST，则在租用期过了 87.5% 时（ T_2 时间到），DHCP 客户必须重新发送请求报文 DHCPREQUEST（重复步骤⑥），然后又继续后面的步骤。

- ⑨ DHCP 客户可以随时提前终止服务器所提供的租用期，这时只需向 DHCP 服务器发送释放报文 DHCPRELEASE 即可。

DHCP 很适合于经常移动位置的计算机。当计算机使用 Windows 操作系统时，点击“控制面板”的“网络”图标就可以找到某个连接中的“网络”下面的菜单，找到 TCP/IP 协议后点击其“属性”按钮，若选择“自动获得 IP 地址”和“自动获得 DNS 服务器地址”选项，就表示是使用 DHCP 协议。

6.7 简单网络管理协议 SNMP

6.7.1 网络管理的基本概念

虽然网络管理还没有精确定义，但它的内容可归纳为：

网络管理包括对硬件、软件和人力的使用、综合与协调，以便对网络资源进行监视、测试、配置、分析、评价和控制，这样就能以合理的价格满足网络的一些需求，如实时运行性能、服务质量等。网络管理常简称为网管。

我们可以看到，网络管理并不是指对网络进行行政上的管理。

网络是一个非常复杂的分布式系统。这是因为网络上有很多不同厂家生产的、运行着多种协议的节点（主要是路由器），而这些节点还在相互通信和交换信息。网络的状态总是不断地变化着。可见，我们必须使用一种机制来读取这些节点上的状态信息，有时还要把一些新的状态信息写入到这些节点上。

下面简单介绍网络管理模型中的主要构件（如图 6-20 所示）。

管理站又称为**管理器**，是整个网络管理系统的核心，它通常是个有着良好图形界面的高性能的工作站，并由网络管理员直接操作和控制。所有向被管设备发送的命令都是从管理站发出的。管理站的所在部门也常称为**网络运行中心 NOC (Network Operations Center)**。管

理站中的关键构件是**管理程序**（如图 6-20 中有字母 M 的椭圆形图标所示）。管理程序在运行时就成为**管理进程**。管理站（硬件）或管理程序（软件）都可称为**管理者(manager)**或**管理器**，所以这里的 manager 不是指人而是指**机器或软件**。网络管理员(administrator)才是指人。大型网络往往实行多级管理，因而有多个管理者，而一个管理者一般只管理本地网络的设备。

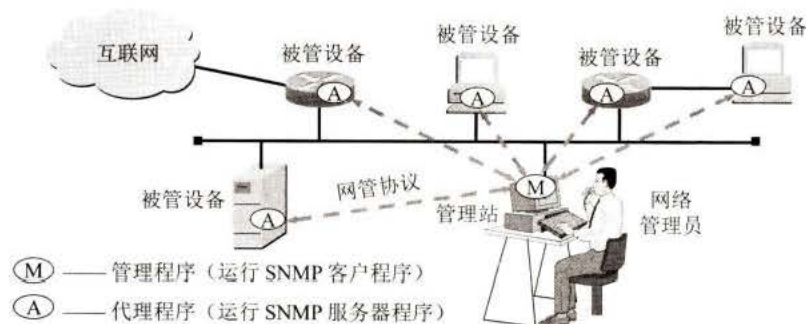


图 6-20 网络管理的一般模型

在被管网络中有很多的**被管设备**（包括设备中的软件）。被管设备可以是主机、路由器、打印机、集线器、网桥或调制解调器等。在每一个被管设备中可能有许多**被管对象(Managed Object)**。被管对象可以是被管设备中的某个硬件（例如，一块网络接口卡），也可以是某些硬件或软件（例如，路由选择协议）的配置参数的集合。被管设备有时可称为**网络元素**或简称为**网元**。在被管设备中也会有一些**不能被管的对象**（在下面的 6.7.2 节将会讲到对象命名树，所谓不能被管的对象就是不在对象命名树上的对象）。

在每一个被管设备中都要运行一个程序以便和管理站中的管理程序进行通信。这些运行着的程序叫作**网络管理代理程序**，或简称为**代理(agent)**（如图 6-20 中有字母 A 的几个椭圆形图标所示）。代理程序在管理程序的命令和控制下，在被管设备上采取本地的行动。

在图 6-20 中还有一个重要构件就是**网络管理协议**，简称为**网管协议**。后面还要讨论它的作用。

简单网络管理协议 SNMP (Simple Network Management Protocol)中的管理程序和代理程序按客户服务器方式工作。管理程序运行 **SNMP 客户程序**，而代理程序运行 **SNMP 服务器程序**。在被管对象上运行的 **SNMP 服务器程序**不停地监听来自管理站的 **SNMP 客户程序**的请求（或命令）。一旦发现了，就立即返回管理站所需的信息，或执行某个动作（例如，把某个参数的设置进行更新）。在网管系统中往往是一个（或少数几个）客户程序与很多的服务器程序进行交互。

关于网络管理有一个基本原理，这就是：

若要管理某个对象，就必然会给该对象添加一些软件或硬件，但这种“添加”对原有对象的影响必须尽量小些。

SNMP 正是按照这样的基本原理来设计的。

SNMP 发布于 1988 年。OSI 虽然在这之前就已制定出许多的网络管理标准，但当时（到现在也很少）却没有符合 OSI 网管标准的产品。SNMP 最重要的指导思想就是要尽可能简单。SNMP 的基本功能包括监视网络性能、检测分析网络差错和配置网络设备等。在网络正常工作时，SNMP 可实现统计、配置和测试等功能。当网络出故障时，可实现各种差错检测和恢复功能。经过近二十年的使用，SNMP 不断修订完善，较新的版本是 SNMPv3，而

前两个版本分别是 SNMPv2 和 SNMPv1。但一般可简称为 SNMP。SNMPv3 最大的改进就是安全特性。也就是说，只有被授权的人员才有资格执行网络管理的功能（如关闭某一条链路）和读取有关网络管理的信息（如读取一个配置文件的内容）。然而 SNMP 协议已相当庞大，一点也不“简单”，整个标准共有八个 RFC 文档[RFC 3411—3418, STD62]。因此这里只能给出一些最基本的概念。

若网络元素使用的不是 SNMP 协议而是另一种网络管理协议，那么 SNMP 协议就无法控制该网络元素。这时可使用委托代理(proxy agent)。委托代理能提供如协议转换和过滤操作等功能对被管对象进行管理。

SNMP 的网络管理由三个部分组成，即 SNMP 本身、管理信息结构 SMI (Structure of Management Information)和管理信息库 MIB (Management Information Base)。下面简述这三部分的作用。

SNMP 定义了管理站和代理之间所交换的分组格式。所交换的分组包含各代理中的对象（变量）名及其状态（值）。SNMP 负责读取和改变这些数值。

SMI 定义了命名对象和定义对象类型（包括范围和长度）的通用规则，以及把对象和对象的值进行编码的规则。这样做是为了确保网络管理数据的语法和语义无二义性。但从 SMI 的名称并不能看出它的功能。请注意，SMI 并不定义一个实体应管理的对象数目，也不定义被管对象名以及对象名及其值之间的关联。

MIB 在被管理的实体中创建了命名对象，并规定了其类型。

为了更好地理解上述的几个组成部分，可以把它们和程序设计进行一下对比。

我们在编程时要使用某种语言，而这种语言就是用来定义编程的规则。例如，一个变量名必须从字母开始而后面接着是字母数字。在网络管理中，这些规则由 SMI 来定义。

在程序设计中必须对变量进行说明。例如，int counter，表示变量 counter 是整数类型。MIB 在网络管理中就做这样的事情。MIB 给每个对象命名，并定义对象的类型。

在编程中的说明语句之后，程序需要写出一些语句用来存储变量的值，并在需要时改变这些变量的值。协议 SNMP 在网络管理中完成这件任务。SNMP 按照 SMI 定义的规则，存储、改变和解释这些已由 MIB 说明的对象的值。

总之，SMI 建立规则，MIB 对变量进行说明，而 SNMP 完成网管的动作。

下面就一一介绍上述的三个构件。

6.7.2 管理信息结构 SMI

管理信息结构 SMI 是 SNMP 的重要组成部分。根据 6.7.1 节所讲，SMI 的功能应当有三个，即规定：

- (1) 被管对象应怎样命名；
- (2) 用来存储被管对象的数据类型有哪些；
- (3) 在网络上传送的管理数据应如何编码。

1. 被管对象的命名

SMI 规定，所有的被管对象都必须处在对象命名树(object naming tree)上。图 6-21 给出了对象命名树的一部分。对象命名树的根没有名字，它的下面有三个顶级对象，都是世界上著名的标准制定单位，即 ITU-T（过去叫作 CCITT）和 ISO，以及这两个组织的联合体，它

们的标号分别是 0 到 2。图中的对象名习惯上用英文小写表示。在 ISO 的下面的一个标号为 3 的节点是 ISO 认同的组织成员 org。在其下面有一个美国国防部 dod (Department of Defense)的子树 (标号为 6)，再下面就是 internet (标号为 1)。在只讨论 internet 中的对象时，可只画出 internet 以下的子树，并在 internet 节点旁边写上对象标识符 1.3.6.1 即可。

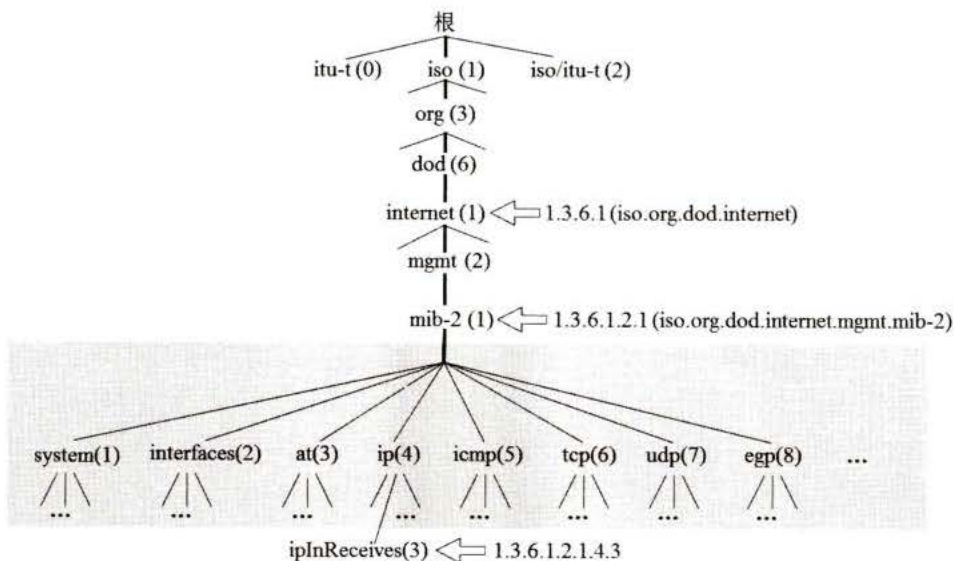


图 6-21 SMI 规定所有被管对象必须在命名树上

在 internet 节点下面的标号为 2 的节点是 mgmt (管理)。再下面只有一个节点，即管理信息库 mib-2，其对象标识符为 1.3.6.1.2.1。在 mib-2 下面包含了所有被 SNMP 管理的对象 (见下面 6.7.3 节的讨论)。

2. 被管对象的数据类型

SMI 使用基本的抽象语法记法 1 (即 ISO 制定的 ASN.1^①) 来定义数据类型，但又增加了一些新的定义。因此 SMI 既是 ASN.1 的子集，又是 ASN.1 的超集。ASN.1 的记法很严格，它使得数据的含义不存在任何可能的二义性。例如，使用 ASN.1 时不能简单地说“一个具有整数值的变量”，而必须说明该变量的准确格式和整数取值的范围。当网络中的计算机对数据项并不都使用相同的表示时，采用这种精确的记法就尤其重要。

我们知道，任何数据都具有两种重要的属性，即值(value)与类型(type)。这里“值”是某个值集合中的一个元素，而“类型”则是值集合的名字。如果给定一种类型，则这种类型的一个值就是该类型的一个具体实例。

SMI 把数据类型分为两大类：简单类型和结构化类型。简单类型是最基本的、直接使用 ASN.1 定义的类型。表 6-4 给出了最主要的几种简单类型。

^① 注：ISO 原以为还会制定出更多的抽象语法记法，但实际上到目前为止并没有第二个抽象语法记法出现。因此 ASN.1 似应写为 ASN。抽象语法只描述数据的结构形式且与具体的编码格式无关，同时也不涉及这些数据结构在计算机内如何存放。